

Text-to-LoRA: Instant Transformer Adaption

Rujikorn Charakorn¹ Edoardo Cetin¹ Yujin Tang¹ Robert T. Lange¹

Abstract

While Foundation Models provide a general tool for rapid content creation, they regularly require task-specific adaptation. Traditionally, this exercise involves careful curation of datasets and repeated fine-tuning of the underlying model. Fine-tuning techniques enable practitioners to adapt foundation models for many new applications but require expensive and lengthy training while being notably sensitive to hyperparameter choices. To overcome these limitations, we introduce Text-to-LoRA (T2L), a model capable of adapting large language models (LLMs) *on the fly* solely based on a natural language description of the target task. T2L is a hypernetwork trained to construct LoRAs in a single inexpensive forward pass. **After training T2L on a suite of 9 pre-trained LoRA adapters (GSM8K, Arc, etc.), we show that the ad-hoc reconstructed LoRA instances match the performance of task-specific adapters across the corresponding test sets.** Furthermore, T2L can compress hundreds of LoRA instances and zero-shot generalize to entirely unseen tasks. This approach provides a significant step towards democratizing the specialization of foundation models and enables language-based adaptation with minimal compute requirements. Our code is available at <https://github.com/SakanaAI/text-to-lora>.

1. Introduction

Biological systems are capable of rapid adaptation, given limited sensory cues. For example, the human visual system can tune its light sensitivity and focus through neuromodulation of the fovea and rod cells (Wurtz et al., 2011; Digre & Brennan, 2012). While recent LLMs exhibit a wide variety of capabilities and knowledge, they remain rigid when

adding task-specific capabilities. In such cases, practitioners often resort to re-training parts of the model (Gururangan et al., 2020; Wei et al., 2021; Dettmers et al., 2022; Tay et al., 2021) using parameter-efficient fine-tuning techniques, e.g., Low-Rank Adaptation (LoRA, Hu et al., 2022). Typically, a LoRA adapter has to be optimized for each downstream task and requires task-specific dataset and hyperparameter setting. This fine-tuning scheme for adaptation significantly limits the possibility of transferring knowledge between tasks and induces engineering overhead. Recently, it has been observed that by inducing structural constraints, the low-rank matrices learned by LoRA adapters can be further compressed. For example, one can train *lossy* versions of the original adapter while maintaining downstream performance (Brüel-Gabrielsson et al., 2024; Kim et al., 2024; Kopiczko et al., 2024). Furthermore, multiple LoRAs can be combined for new tasks at inference time (Ostapenko et al., 2024). At the core of these approaches lies the explicit use of decomposition or dimensionality reduction techniques (e.g., SVD or routing) for better compression and online composition of existing LoRAs. This raises the following questions:

1. Can we end-to-end train a neural network to compress many pre-trained LoRAs?
2. Can we decode new task-specific LoRA adapters solely based on natural-language instructions for an unseen task at test time?

We hypothesize that different LoRA adapters share the same underlying adaptation mechanism and can be optimized simultaneously without any explicit structure or recipe for combining them. To explicitly test this hypothesis, we propose T2L, a hypernetwork (Ha et al., 2016) that compresses task-specific LoRAs and generates new LoRA adapters *zero-shot* at inference time. T2L is trained to compress LoRAs on a diverse task distribution from the Super Natural Instructions (SNI) dataset (Wang et al., 2022). Importantly, T2L takes a natural language description of the target task as an input, allowing zero-shot LoRA generation to unseen tasks. Empirically, we show that T2L can effectively be trained either to reconstruct pre-trained adapters or via supervised fine-tuning on a distribution of downstream tasks (see Fig-

¹Sakana AI. Correspondence to: Rujikorn Charakorn <rujjikorn@sakana.ai>, Robert T. Lange <robert@sakana.ai>.

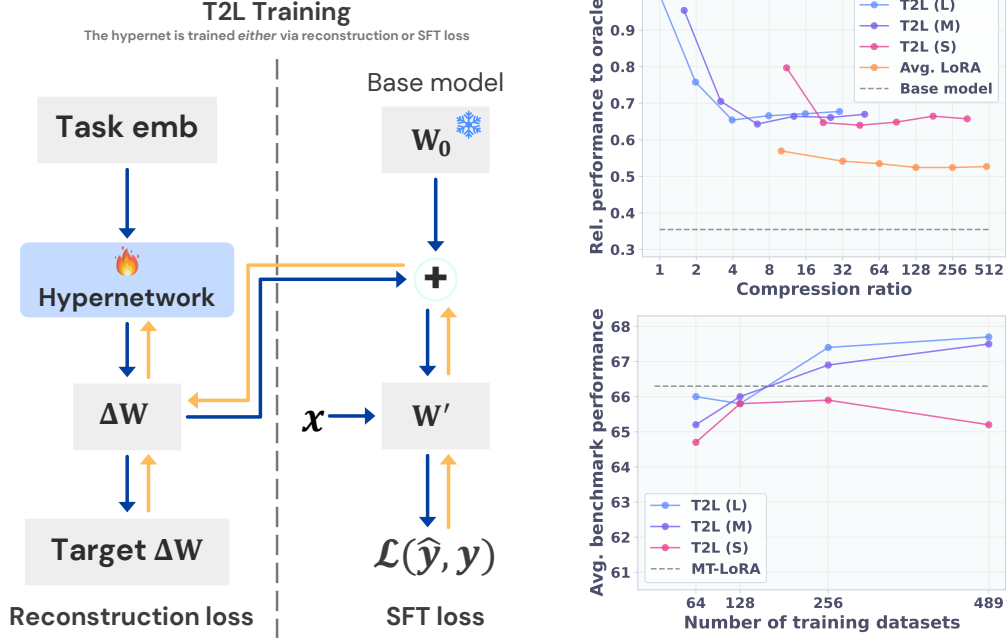


Figure 1: **Left:** Conceptual overview of **T2L**’s training routine. Given a set of task description embeddings, we train a hypernetwork to generate LoRA adaptation matrices (ΔW) for various tasks. The weights of **T2L** are *either* optimized to distill pre-trained LoRA weights or via multi-task supervised fine-tuning on downstream tasks. **Right, Top:** Relative performance to the oracles on training SNI tasks with varying compression ratios. **Right, Bottom:** Zero-shot LoRA generation performance on 10 benchmark tasks. As we increase the number of pre-training datasets, the performance of **T2L** increases for 3 different **T2L** architectures.

ure 1, top right). After training, **T2L** outperforms a multi-task LoRA baseline and Arrow Routing (Ostapenko et al., 2024), a state-of-the-art zero-shot LoRA routing method, on various benchmark tasks. Furthermore, we show that **T2L** can generate LoRA adapters for previously unseen tasks solely using the language-based task description. This result highlights the generalization capabilities and applicability of our proposed indirect adaptation encoding. Our contributions are summarized as follows:

1. We introduce hypernetwork-based architectures for producing LoRA adapters with a single forward pass (Section 3) based on text descriptions. **T2L** architectures can be trained using both distillation of pre-trained adapters and supervised multi-task fine-tuning.
2. We show that **T2L** can efficiently encode hundreds of LoRA adapters (Section 4). While the compression is lossy, **T2L** maintains the performance of task-specifically tuned LoRA adapters. Furthermore, **T2L** can generalize to unseen tasks given suitable natural language descriptions of the tasks.
3. We provide rigorous ablations (Section 5) including **T2L** scaling with datasets (see Figure 1, bottom right),

the impact of different task description embeddings, the training routines, and text-based task descriptions.

4. Finally, we analyze the nature of **T2L** generations. We find semantically meaningful LoRA clusters when visualizing the generated LoRAs in a dimensionality-reduced space (Section 5.5). Furthermore, we study the relationship between LoRA adapters and find compelling evidence why reconstruction-trained **T2L** cannot generalize (Appendix D).

2. Preliminaries

We utilize multiple fine-tuning datasets $\mathcal{D} = \{\mathcal{D}^1, \dots, \mathcal{D}^T\}$, which correspond to different tasks $\mathcal{T} = \{t^1, \dots, t^T\}$. For the purpose of training **T2L**, we assume that each fine-tuning dataset has a set of natural language *task descriptions* ($Z^i = \{z_1^i, \dots, z_m^i\}$): $\mathcal{D}^i = \{X^i, Y^i, Z^i\}$. The task descriptions do not need to be specific to each sample but rather a general description of the dataset. For a single task t^i , the fine-tuning objective of an LLM with pre-trained weights (Ψ) is given by

$$\Delta W^i = \underset{\Delta W^i}{\operatorname{argmin}} \mathcal{L}_{\text{SFT}}(\mathcal{D}^i, \Psi, \Delta W^i), \quad (1)$$

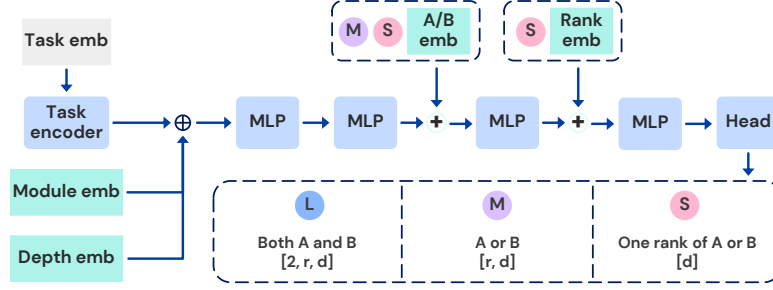


Figure 2: Overview of T2L architectural variations. The dashed box at the bottom shows the output size of a single forward pass of T2L. Blue boxes are trainable modules. Cyan boxes are trainable embedding layers. Components in dashed boxes are only used with their corresponding architectures. r is the rank of a LoRA adapter and d is the size of the input and the output dimension.

where $\mathcal{L}_{\text{SFT}}$ gives the supervised fine-tuning loss and ΔW^i is the fine-tuning adaption for task t^i to the base weights. For the *multi-task* setting, we train a single adapter ΔW to minimize the expected loss over the union of all datasets $\mathcal{D}$:

$$\Delta W = \underset{\Delta W}{\operatorname{argmin}} \mathbb{E}_{\mathcal{D}^i \sim \mathcal{D}} \mathcal{L}_{\text{SFT}}(\mathcal{D}^i, \Psi, \Delta W). \quad (2)$$

Low-Rank Adaptation (LoRA, Hu et al., 2022): LoRA is a parameter-efficient fine-tuning method that freezes the pre-trained weights of a base model and only learns low-rank weight matrices, which serve as an adapter to the base model. For each selected linear transformation $h = W_0 x$, the fine-tuned transformation is given by $h = W_0 x + \Delta W x = W_0 x + B^T A x$, where $A, B \in \mathbb{R}^{r \times d}$ are weight matrices of rank $r < d$. We omit the layer index and module type of the LoRA weights when referring to all LoRA weights. Otherwise, we use subscripts to represent the layer index and module type, e.g., $\Delta W_{m,l}$, where m is the module type (e.g., query projection) and l is the layer index.

Hypernetworks: A hypernetwork is a neural network that generates parameters for another ‘base’ network (Ha et al., 2016). It serves as an indirect encoding (Schmidhuber, 1997; Stanley & Miikkulainen, 2003; Zhang et al., 2018; Schug et al., 2024) of the base network, given that the parameter count of the hypernetwork is much smaller. This compression is achieved by learning to share parameters indirectly. More specifically, given a layer-specific descriptor vector ϕ_l , a hypernetwork with parameters θ generates the parameters of the base model at layer $l \in \{1, \dots, L\}$ as follows: $W_l = h_\theta(\phi_l)$. Traditionally, the layer descriptors are either one-hot or learned vectors. The weights θ are then trained via end-to-end optimization on a downstream task.

3. Text-to-LoRA: Learning to Compress and Generate LoRAs

In this work, we utilize a hypernetwork to generate LoRA adapters for task-specific adaptation. For each target module (m) and layer index (l), a hypernetwork generates the two

low-rank matrices A, B based on a task description $z^i \in Z^i$ of a task t^i as follows:

$$\Delta W_{m,l}^i = h_\theta(\phi_{m,l}^i), \text{ with} \quad (3)$$

$$\phi_{m,l}^i = \text{concat} [f(z^i), E[m], E[l]], \quad (4)$$

where f gives a vector representation of a text description, typically represented by a CLS token of a bidirectional transformer model or last token activation of an LLM. E is a learnable embedding dictionary indexed by either a module type m or a layer index l . For legibility, we introduce a shorthand notation for T2L’s output $\Delta W^i := h_\theta(\phi^i) := h_\theta(\{\phi_{m,l}^i\})$. Then, a supervised fine-tuning training objective for T2L is

$$\theta = \underset{\theta}{\operatorname{argmin}} \mathbb{E}_{\mathcal{D}^i \sim \mathcal{D}, z^i \in Z^i} \mathcal{L}_{\text{SFT}}(\mathcal{D}^i, \Psi, h_\theta(\phi^i)), \quad (5)$$

Note that values of m and l can be batched, which allows T2L to generate ΔW for all modules and layer indices efficiently within a single forward pass.

3.1. Text-to-LoRA Architectures

Most of a hypernetwork’s parameters come from the output layer, which scales linearly with the size of the target weights (Von Oswald et al., 2019). To explore the complexity-performance trade-off, we propose three variants of T2L: **L**, **M**, and **S**. We impose different output spaces on the hypernetwork that represent different inductive biases and parameter counts (see Figure 2). We note that all variants use the same backbone architecture and only differ in their output heads and learnable embeddings. The **L architecture** is the largest variant. Its final linear layer outputs low-rank A and B matrices simultaneously, with the number of weight connections to the output head $|\theta_{\text{head}}| = d_{\text{out}} \times 2 \times r \times d$, where d_{out} is the output size of the last MLP block. **M architecture** is the medium-sized model with a shared output layer between the low-rank A and B matrices. That is, the head outputs a low-rank

matrix, either A or B , depending on the learnable embedding. The size of the output head is $|\theta_{\text{head}}| = d_{\text{out}} \times r \times d$. Finally, **S architecture** is the most parameter-efficient model with the strongest inductive biases, where the hypernetwork outputs only one rank of a low-rank matrix at a time. This output space makes the size of the head much smaller: $|\theta_{\text{head}}| = d_{\text{emb}} \times d$. For reference, a LoRA adapter has $r \times d \times 2 \times L \times |M|$ trainable parameters, where L is the number of layers and $|M|$ is the number of target modules. The default value of d_{out} is 512. We note that every architecture can generate all the low-rank matrices A and B in a single forward pass by batching all the input embeddings. We provide more details of the architectures in Appendix F and the weight initialization method that leads to stable training in Appendix G.

3.2. Training Text-to-LoRA via LoRA Reconstruction

The most straightforward way to train **T2L** is to reconstruct pre-trained task-specific LoRAs. This setup allows us to utilize publicly available libraries of LoRAs (Brüel-Gabrielsson et al., 2024; Zhao et al., 2024). Alternatively, one can also use a two-stage procedure, in which a library of LoRAs is pre-trained in the first stage and then train **T2L** to reconstruct them. For the sole purpose of compressing LoRAs, we can train **T2L** using one-hot or learnable vectors as task embeddings. However, these embeddings do not allow zero-shot LoRA generation for unseen tasks. To enable zero-shot LoRA generation, we additionally condition **T2L** with embeddings of natural language task descriptions, which allows **T2L** to generate LoRA adapters for various tasks—including unseen ones—given corresponding task descriptions. Given a suitable library of LoRA adapters Ω , the reconstruction loss for **T2L** can be written as

$$\mathcal{L}(\Omega, \theta) = \mathbb{E}_{\Delta W^i \sim \Omega} |\Delta W^i - h_{\theta}(\phi^i)|. \quad (6)$$

3.3. Training Text-to-LoRA via Supervised Fine-Tuning

Alternatively, **T2L** can be directly optimized on fine-tuning datasets. Training **T2L** with SFT sidesteps the need for intermediate target LoRA adapters and allows for end-to-end training. This training scheme is preferred if existing trained LoRAs are not naturally clustered by their functionalities or downstream tasks. For instance, t^1 and t^2 could be two related tasks requiring a similar LLM capability, but ΔW^1 and ΔW^2 could be in different minima. Thus, **T2L** trained via reconstruction training would have to compress numerically different ΔW^1 and ΔW^2 , making it less likely to generalize. In fact, we empirically find that a **T2L** trained via reconstruction fails to generalize to unseen tasks (Section 5.4). In contrast, an SFT-trained **T2L** can implicitly learn to cluster tasks, which has been shown to improve zero-shot LoRA routing performance (Ostapenko et al., 2024). The SFT loss for **T2L** is given by Equation (5).

4. Experiments

We investigate the effectiveness of the different **T2L** architectures and training schemes in terms of the compression of adapters (Section 4.1) and zero-shot LoRA generation for unseen tasks (Section 4.2). As baselines, we consider task-specific LoRAs, element-wise averaged LoRA, and multi-task LoRA—a LoRA adapter trained on all training tasks. We also implement Hyperdecoders (Iverson & Peters, 2022)—a hypernetwork that generates LoRAs on a per-sequence basis—based on our proposed architectures. To boost the performance of the base models without fine-tuning, we utilize few-shot in-context learning (ICL, Brown et al., 2020; Dong et al., 2024) and task description pre-pending, i.e., providing task description at the beginning of each query. Additionally, we include results of Arrow Routing zero-shot performance from Ostapenko et al. (2024). Note that the performance can only be compared indirectly as it uses a different set of LoRA adapters and training tasks. Furthermore, there are likely differences in the benchmark evaluation prompts.

In most experiments, we use Mistral-7B-Instruct (Jiang et al., 2023) as the base LLM model except in Tables 7 and 8 where Llama-3.1-8B-Instruct and Gemma-2-2b-Instruct are used as the base models, respectively. We use gte-large-en-v1.5 (Li et al., 2023; Zhang et al., 2024) for extracting the task embedding from a natural language task description. All LoRA adapters are of rank 8 and only target the query and the value projection modules in every attention block of the base LLM (totaling 3.4M parameters). With this LoRA configuration, **L**, **M**, and **S** have 55M, 34M, and 5M trainable parameters respectively.

We utilize the SNI dataset (Wang et al., 2022) for training LoRA adapters. We use a subset of 500 tasks following Brüel-Gabrielsson et al. (2024). We use 11 tasks for hold-out validation and removed 10 datasets due to data contamination from the evaluation benchmark tasks, leaving 479 datasets for training. All samples are in English. More details of the datasets can be found in Appendix J. For evaluation, we choose 10 widely used benchmarks that collectively cover a variety of LLM capability assessments, e.g., reasoning, math, science, coding, and world knowledge. Specifically, we include the following benchmarks: Arc-challenge (ArcC) and Arc-easy (ArcE) (Clark et al., 2018), BoolQ (Clark et al., 2019), GSM8K (Cobbe et al., 2021), Hellaswag (HS) (Zellers et al., 2019), OpenBookQA (OQA) (Mihaylov et al., 2018), PIQA (Bisk et al., 2020), Winogrande (WG) (Keisuke et al., 2019), HumanEval (HE) (Chen et al., 2021), and MBPP (Austin et al., 2021).¹ Task

¹The benchmark tasks share some similarities with the training tasks. Specifically, they are mostly multiple-choice question-answering tasks. Also, there are similar and overlapping domains

Table 1: Benchmark performance of **T2L** trained via reconstruction loss on 9 benchmark tasks. **Green highlight** indicates that **T2L** outperforms the benchmark-specific LoRA adapters.

	ArcC (acc)	ArcE (acc)	BQ (acc)	GSM8K (acc)	HS (acc)	OQA (acc)	PIQA (acc)	WG (acc)	MBPP (pass@1)	Avg. (9 tasks)
Base model	65.4	77.8	71.6	40.9	49.7	54.2	72.8	45.0	43.1	55.8
One-Hot Task E.										
T2L (Recon) L	76.4	89.9	89.4	53.8	92.6	85.0	69.7	51.2	52.6	73.4
T2L (Recon) M	76.7	89.9	89.4	53.2	92.6	85.0	69.9	51.4	52.9	73.4
T2L (Recon) S	75.2	88.8	87.4	50.9	89.1	75.6	83.9	58.1	48.1	73.0
Task Description E.										
T2L (Recon) L	76.6	89.8	89.4	53.9	92.6	85.0	69.6	51.2	51.8	73.3
T2L (Recon) M	76.5	89.9	89.4	53.9	92.5	84.9	70.4	51.6	52.8	73.5
T2L (Recon) S	75.4	88.8	87.8	49.1	89.7	76.7	84.2	56.9	48.0	73.0
Task-specific LoRAs	76.6	89.9	89.4	53.5	92.6	85.0	69.9	51.1	52.1	73.3

descriptions for the training datasets and the benchmarks are fully generated, as described in Appendix L. When we use a language task embedding as a part of the input, we average **T2L** performance using three descriptions for each benchmark.

4.1. LoRA Compression

In this experiment, we aim to investigate whether **T2L** can recover the performance of trained LoRAs via reconstruction training. For quality control and consistent evaluation, we train a task-specific LoRA (oracle) on the training split of each benchmark task, collectively forming a library of LoRAs. Table 1 shows the benchmark performance of **T2L** trained by distilling 9 benchmark-specific LoRAs using either one-hot or natural language task embeddings from gte-large-en-v1.5. We note that the benchmark tasks are indirectly seen during training by **T2L**, as it learns to distill benchmark-specific LoRAs. We can see that **T2L** fully recovers the performance of the oracle adapters with both task embedding types. Notably, **T2L** outperforms task-specific LoRAs on several benchmarks (highlighted in green). We hypothesize that the gain comes from the lossy compression of the target LoRAs, which acts as a regularization on the already trained LoRA weights. This effect is most apparent on PIQA and WG benchmarks, where the oracle LoRA overfits and performs worse than the base model.

Next, we explore whether **T2L** conditioned on one-hot task vectors can maintain the oracle single-task LoRAs’ performance when using an increasing number of training tasks. Figure 3 shows the performance of one-hot **T2L** on the test splits of a subset of 10 SNI training tasks with varying

between the two splits. For example, the ARC benchmarks are similar to SNI task #47. However, some benchmarks are very different from the training distribution, e.g., MBPP and HumanEval, as the training tasks do not contain any code generation tasks.

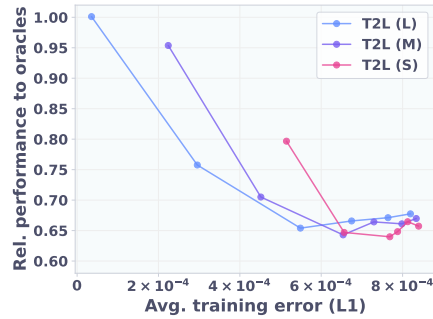


Figure 3: Relative performance and training reconstruction error of **T2L** instances trained with an increasing number of tasks ({16, 32, 64, 128, 256, 479} tasks from left to right).

degrees of final average training L1 reconstruction error. We train various **T2L** instances for each architecture using {16, 32, 64, 128, 256, 479} training tasks, leading to an effective increase in the training reconstruction error. Although **T2L** fully recovers the oracles’ performance when the reconstruction loss is less than 10^{-4} , the performance drops as the training error increases. This result suggests that **T2L** learns a lossy compression of the target LoRAs. Still, we find that all **T2L** architectures can maintain around 65% of oracles’ performance, and the performance does not drop further even at $> 8 \times 10^{-4}$ per-element L1 error. Despite the performance drop, we show that increasing the number of training tasks is beneficial in the SFT setup, increasing zero-shot benchmark performance of **T2L** in Section 5.1.

4.2. Zero-Shot LoRA Generation

Here, we explore whether **T2L** can generate useful LoRA adapters for unseen tasks. We train **T2L** with SFT on 479 SNI tasks, each with 128 task descriptions. For each data point in a training minibatch, we sample a description from the corresponding dataset in an online fashion. Table 2

Table 2: Zero-shot performance on unseen benchmark tasks. SFT-trained T2L generates LoRAs based on unseen task descriptions. Its performance is an average of three generated LoRAs, each with a different instance of task descriptions. Arrow Routing results are taken from Ostapenko et al. (2024). **Green highlight** indicates higher performance than that of the benchmark-specific LoRA adapters. **Bold numbers** are used when the performance is higher than the multi-task LoRA.

	ArcC (acc)	ArcE (acc)	BQ (acc)	HS (acc)	OQA (acc)	PIQA (acc)	WG (acc)	MBPP (pass@1)	Avg. (8 tasks)	GSM8K (acc)	HE (pass@1)	Avg. (10 tasks)
No Test-Time Adaptation												
Mistral-7B-Instruct	65.4	77.8	71.6	49.7	54.2	72.8	45.0	43.1	60.0	40.9	37.2	55.8
Prepending task desc.	72.0	85.8	67.6	58.9	63.4	77.9	59.0	41.6	65.8	40.9	39.0	60.6
3-shot ICL	72.1	85.9	71.7	59.0	66.2	76.2	58.0	42.6	66.5	40.9	37.2	61.0
Average LoRA	70.7	84.4	75.4	59.9	59.0	78.0	54.3	47.1	66.1	42.4	37.8	60.9
Multi-task LoRA	76.2	88.3	85.5	65.2	68.0	81.8	62.4	48.1	71.9	47.5	39.6	66.3
Zero-Shot Adaptation												
Arrow Routing	60.9	86.2	87.6	80.8	48.6	83.0	68.5	50.2	70.7	N/A	28.7	N/A
Hyperdecoders (per-instance)	76.6	88.5	83.9	65.2	76.6	81.3	64.9	51.6	73.6	43.6	40.9	67.3
T2L (SFT) S	76.0	88.7	83.8	68.0	71.6	82.3	61.0	41.2	71.6	47.3	39.0	65.9
T2L (SFT) M	77.2	89.0	84.3	65.1	76.1	81.8	64.0	50.5	73.5	45.2	41.3	67.5
T2L (SFT) L	77.5	88.9	85.0	66.5	75.5	82.1	64.2	51.9	73.9	45.8	39.2	67.7
Oracle												
Task-specific LoRAs	76.6	89.9	89.4	92.6	85.0	69.9	51.1	52.1	75.8	53.5	N/A	N/A

shows the zero-shot performance on 10 benchmark tasks. Here, we present the best model of each variant from our scaling experiment in Section 5.1. We observe that a multi-task LoRA adapter performs well on the benchmarks despite no additional fine-tuning. Still, there is a performance gap between task-specific LoRAs and MT LoRA. We observe that SFT-trained T2L indeed generates useful LoRAs, thus improving over the multi-task LoRA adapter consistently and across benchmarks (indicated by bold numbers). Notably, even though T2L cannot fully bridge the performance gap with task-specific LoRAs, it outperforms the oracles on a subset of tasks (highlighted in green). We further investigate the generality of our proposed method with different base models including Llama (Dubey et al., 2024) and Gemma (Team et al., 2024) models in Appendix A. We note that one of the main advantages of T2L is its efficiency. To emphasize T2L’s efficiency, we provide an ad-hoc FLOPs analysis in Appendix I.

5. Ablations and Analyses

5.1. Increasing Training Compute Proportional to the Number of Training Tasks

In this section, we explore the scalability of T2L by varying the training tasks and scale the training budget proportionally to the dataset size on all variants. Table 3 shows that, after increasing the number of training tasks and compute budget, T2L generally benefits from the additional training tasks. However, **S** does not benefit from extended training with 479 tasks, potentially due to its limited model capacity. We additionally investigate the effect of the task diversity on the robustness of T2L by training on more tasks without

scaling the training budget in Appendix C. We find that it is crucial to scale the compute budget according to the number of training tasks. For instance, **M** with scaled compute budget improves over training runs with a fixed budget when using 256 or more training tasks.

5.2. Task Embedding Models

Table 4 shows the zero-shot benchmark performance with two different embedding models: gte-large-en-v1.5 and Mistral-7B-Instruct. For the gte model, we extract a task description by using the activation of the CLS token in the last layer as the model is a bidirectional model. For Mistral, we use the activation of the last token in the sequence to represent a given description (BehnamGhader et al., 2024). Table 4 shows the results with the two embedding models used for T2L SFT training on 128 tasks. Both embedding models yield T2L instances with comparable generalization capability, suggesting T2L’s robustness to task description embedding methods.

5.3. Varying Task Descriptions

We investigate the impact of task descriptions on the performance of generated LoRAs using four types of descriptions:

- **Train:** Training descriptions of corresponding tasks.
- **Eval:** Unseen descriptions of corresponding tasks.
- **Random strings:** Random literal strings.
- **Train (random):** Training descriptions randomly sampled from other tasks.

Table 3: Performance of SFT-trained T2L with varying numbers of training tasks.

	Number of tasks	Max SGD steps	ArcC (acc)	ArcE (acc)	BQ (acc)	GSM8K (acc)	HS (acc)	OQA (acc)	PIQA (acc)	WG (acc)	HE (pass@1)	MBPP (pass@1)	Avg.
T2L (SFT) L	479	1M	77.5	88.9	85.0	45.8	66.5	75.5	82.1	64.2	39.2	51.9	67.7 ▲
	256	640K	77.3	88.1	84.3	46.0	64.5	75.7	81.9	64.0	39.8	52.1	67.4 ▲
	128	320K	76.6	88.4	85.2	46.1	67.0	74.3	81.6	55.0	38.2	45.7	65.8 ▼
	64	160K	75.5	88.0	84.5	43.9	65.5	70.7	80.5	59.5	39.8	51.7	66.0
T2L (SFT) M	479	1M	77.2	89.0	84.3	45.2	65.1	76.1	81.8	64.0	41.3	50.5	67.5 ▲
	256	640K	75.9	89.3	85.0	47.0	65.3	73.7	81.6	63.2	39.8	48.6	66.9 ▲
	128	320K	74.9	88.3	85.5	44.9	64.8	72.8	80.7	61.6	42.9	43.5	66.0 ▲
	64	160K	73.6	87.7	84.5	43.2	64.6	70.5	79.9	56.0	40.7	51.4	65.2
T2L (SFT) S	479	1M	77.7	88.3	85.0	46.3	65.3	73.9	82.4	61.9	34.6	36.6	65.2 ▼
	256	640K	76.0	88.7	83.8	47.3	68.0	71.6	82.3	61.0	39.0	41.2	65.9 ▲
	128	320K	74.9	88.0	84.5	44.4	66.2	72.2	82.0	59.3	39.0	47.3	65.8 ▲
	64	160K	75.4	88.4	85.0	43.1	64.8	70.7	81.5	51.6	39.4	46.7	64.7

Table 4: Zero-shot benchmark performance of SFT T2L trained on 128 tasks using different text embedding models.

	gte			Mistral		
Avg. Benchmark performance	S	M	L	S	M	L
	65.8	66.0	65.8	64.7	66.2	66.0
Avg.	65.9			65.6		

Table 5: T2L trained via reconstruction on 9 tasks performs well when given aligned task descriptions. Unaligned descriptions produce lower benchmark performance.

	Aligned		Unaligned	
	Train	Eval	Train (random)	Random strings
T2L L	73.3	73.6	49.1	68.2
T2L M	73.5	70.2	49.5	68.5
T2L S	73.0	72.9	55.7	53.9
Avg.	73.3	72.2	51.4	63.5

For each description type, we use the gte-large-en-v1.5 embedding and report the average performance using three descriptions. The four types can be grouped into two categories based on the alignment between the descriptions and the tasks: aligned (**Train, Eval**) and unaligned (**Train (random)** and **Random strings**). Note that we use reconstruction-trained T2L in this experiment. That is, the hypernetwork has seen training descriptions of the benchmarks during training. We observe a performance gap between the two description categories. Specifically, training and evaluation descriptions generate the best performing LoRAs, matching the performance of oracle LoRAs, despite the evaluation descriptions being unseen. These results suggest that T2L is robust to changes in the task description as long as the descriptions are aligned with the task. On the other hand, if the descriptions are not aligned with the task at hand, the generated LoRAs will not perform as well, as indicated by the performance

Table 6: Reconstruction vs SFT training scheme.

Benchmark performance	Recon			SFT		
	S	M	L	S	M	L
	61.8	61.7	62.0	64.8	66.5	67.5
Avg.	61.8			66.3		

of the unaligned group. We believe that using an LLM for adjusting the description alignment could effectively sidestep this failure case of T2L. Additionally, we provide a qualitative result demonstrating steerability and an unsuccessful example of T2L in Figure 4. Importantly, the last two examples in Figure 4 (iii, iv) are both correct but have different answer styles thanks to different descriptions. We remark that Hyperdecoders (Iverson & Peters, 2022) cannot exhibit such steerability as it uses the problem instance as the input to the hypernetwork.

5.4. Training Schemes

In this section, we investigate the zero-shot performance of SFT-trained and reconstruction-trained T2L. All model instances are trained with roughly equal wall-clock time of 10 hours (see Appendix H for details). From Table 6, we can see a clear performance gap between reconstruction and SFT training schemes. Specifically, SFT produces T2L instances that perform significantly better than those trained via reconstruction (66.3 vs 61.83 benchmark performance averaged over model architectures). We attribute the performance difference to the library of LoRAs needed for reconstruction training. For reconstruction-trained T2L to generalize, the target LoRA adapters of similar tasks should be clustered in some latent manifold. In contrast, SFT training does not need pre-trained task-specific LoRA adapters, thus sidestepping this challenge via end-to-end learning. In Appendix D, we show that pre-trained adapters for similar tasks do not live nearby in the weight space, supporting our claim of a

User Input

Toulouse has twice as many sheep as Charleston. Charleston has 4 times as many sheep as Seattle.
How many sheep do Toulouse, Charleston, and Seattle have together if Seattle has 20 sheep?

(i) Base model response

Great question! Let's break it down step by step:

1. We know that Seattle has 20 sheep.
2. Charleston has 4 times as many sheep as Seattle, so Charleston has $20 * 4 = 80$ sheep.
3. Toulouse has twice as many sheep as Charleston, so Toulouse has $80 * 2 = 160$ sheep.
4. To find the total number of sheep in all three cities, we add the number of sheep in each city: 160 (Toulouse) + 80 (Charleston) + 20 (Seattle) = 240 sheep.

So, Toulouse, Charleston, and Seattle together have **240 sheep**.

**(ii) Generate LoRA using the following (low-quality) description:
"solve this task please"**

Charleston has 4 times as many sheep as Seattle, and Seattle has 20 sheep, so Charleston has $20 * 4 = 80$ sheep.
Toulouse has twice as many sheep as Charleston, so Toulouse has $80 * 2 = 160$ sheep.
Toulouse, Charleston, and Seattle have sheep together: $160 + 80 + 20 = 240$ sheep.

(iii) Generate LoRA using the following description:

"This task challenges your problem-solving abilities through mathematical reasoning. You must carefully read each scenario and systematically work through the data to compute the final outcome."

Toulouse has twice as many sheep as Charleston, so Toulouse has $2 * \text{Charleston} = 2 * 4 = 8$ times as many sheep as Seattle.
Seattle has 20 sheep, so Toulouse has $8 * 20 = 160$ sheep.
Charleston has 4 times as many sheep as Seattle, so Charleston has $4 * 20 = 80$ sheep.
Toulouse, Charleston, and Seattle have sheep together = 160 (Toulouse) + 80 (Charleston) + 20 (Seattle) = **260 sheep**.

(iv) Generate LoRA using the following description:

"Use your programming skill to think algorithmically, logically and systematically. Carefully read each scenario and systematically work through the data to compute the final outcome. Use your math skill to calculate correctly."

let me check the relationships between the number of sheep in each city:

1. Toulouse has twice as many sheep as Charleston.
2. Charleston has 4 times as many sheep as Seattle.

From point 2, we know that Charleston has $4 * 20 = 80$ sheep. From point 1, Toulouse has $2 * 80 = 160$ sheep.
So, Toulouse, Charleston, and Seattle have a total of $160 + 80 + 20 = 260$ sheep.

Figure 4: Qualitative examples of responses from applying LoRA generated by T2L to the *Mistral-7B-Instruct* base model on a GSM8K problem instance. (i) The response from the base model is incorrect. (ii) Applying a LoRA generated from a *low-quality* task description does not make the model output the correct response. (iii, iv) Descriptions that are aligned with the problem lead to generated LoRAs that steer the base model to output correct responses. Descriptions from (iii) and (iv) influence the model to generate different reasoning paths, highlighting the steerability of T2L.

potential problem when reconstructing pre-trained LoRAs.

5.5. Visualization of T2L Activations

Next, we aim to understand T2L further and see whether it generates task-specific LoRA adapters for unseen tasks with unseen descriptions. We probe SFT T2L M trained on 256 training tasks in the zero-shot evaluation setting. The model is probed on all the benchmark tasks, each with three unseen descriptions. Figure 5 shows the 2D t-SNE projection of T2L's task encoder activations and the outputs of the last MLP block. We can see clear clustering in both projection plots based on the tasks (colors and shapes). T2L generates different adapters for different tasks, confirming that T2L indeed performs task-specific adaptation 'on the fly'. Moreover, similar tasks, e.g., MBPP and HumanEval, are clustered together in both plots, suggesting that SFT-trained T2L produces similar adapters for semantically similar tasks.

6. Related Work

Hypertextworks for Adaptation: Hypertextworks (Ha et al., 2016) provide a general indirect encoding method for neural network weights. They have been applied to different architectures (e.g., in attention, Schug et al., 2024) and training paradigms (e.g., in continual learning, Von Oswald et al., 2019). Here, we focus on generating low-rank adapters using natural language instructions. Previous work (Mahabadi et al., 2021; He et al., 2022; Ortiz-Barajas et al., 2024) considers hypertextworks for LLM adaptation in a multi-task context but only uses learned task identifiers instead of natural language for adaptation. Thus, these approaches do not enable task-wise zero-shot generalization.

Hypertextworks for Zero-Shot LLM Adaptation: Xiao et al. (2023) explore the use of hypertextworks on a limited set of English dialects; they only consider five dialects, one of which is unseen. Furthermore, the hypertextwork relies on an expert-based transformation of the dialects, limiting the possibility for generalization. Mu et al. (2024) propose

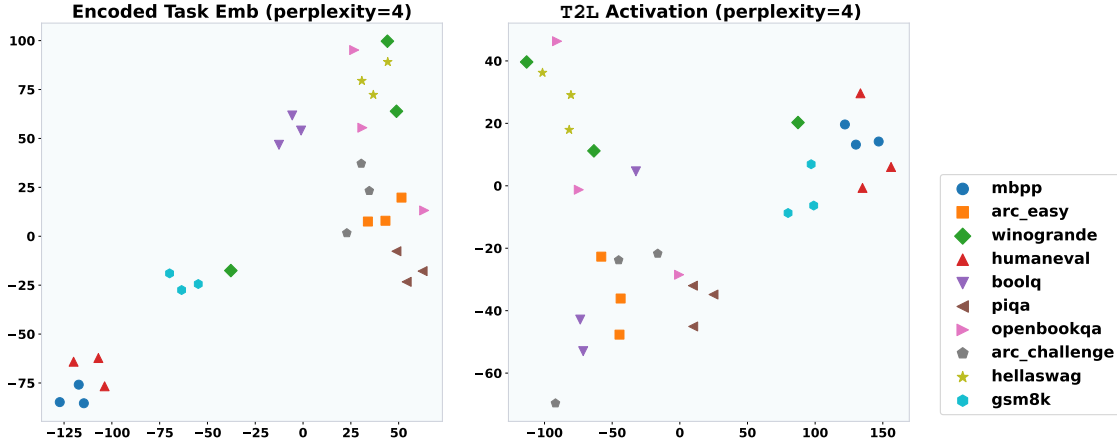


Figure 5: 2D t-SNE projection of activations of **T2L**’s task encoder (left) and activations of the last MLP block (right) grouped by benchmark tasks (represented by colors). We probe **T2L** with unseen three task descriptions per benchmark. We can see activations clustering in both plots, indicating that **T2L** indeed learns to generate LoRAs tailored to specific tasks.

Gisting, a method that learns to compress an in-context task description to prefix tokens, allowing the language model to follow instructions with fewer tokens. However, Gisting is limited to prefix tokens—only influencing the attention matrices of the base model. Thus, prefix tokens are less flexible compared to LoRAs that can modify different parts of LLMs, e.g., attention blocks. Hyperdecoders (Iverson & Peters, 2022) is a hypernetwork that generates adapters on the fly based on the input sequence. While per-sequence adaptation is desirable for benchmark evaluation—where the LLM should always output the correct answer—we argue that description-based adaptation gives more control to users since they can steer the LLM in creative ways based on user-generated descriptions (see Figure 4). Furthermore, the generated adapters cannot be efficiently fused into the base model, leading to significant overhead for each query.

Closely related to our work are HyperTuning (Phang et al., 2023), HNET-LM (Deb et al., 2022), and HINT (Iverson et al., 2023). Differing from prior work that heavily focuses on pre-trained encoder-decoder models, e.g., T5 (Raffel et al., 2020) or BART (Lewis, 2019), we use frontier instruction fine-tuned models as the base models, i.e., Mistral, Llama, Gemma. Also, prior work typically relies on initializing a part of their hypernetworks from the base model (e.g., tying task encoder’s weights to the base model) to achieve good performance or stable training as opposed to ours that are task-embedder agnostic and can freely change the task embedding model (Section 5.2). Additionally, our work utilizes generated descriptions instead of the ones provided by the SNI dataset. In Appendix B, we show that using generated descriptions increase the performance of **T2L** considerably. Overall, our work improves upon prior work in several ways, including achieving task-wise zero-shot generalization on various frontier instruction-tuned language models, simpler and more general hypernetwork

input requirements, investigation of training regimes, and more comprehensive experiments, ablations, and analyses.

Concurrent to our work, Lv et al. (2024) propose a similar approach that utilizes a hypernetwork to generate LoRA adapters at inference time. However, their hypernetwork assumes that the context vector provided to the hypernetwork contains few-shot examples. In contrast, **T2L** only assumes a task description, which users can produce themselves within seconds.

7. Discussion and Limitations

Discussion. We rely on generated descriptions from GPT-4o mini to ensure high-quality and consistent task descriptions. It is plausible that when **T2L** is deployed in real-world scenarios, users might not input high-quality descriptions, which could cause performance degradation of generated adapters. Our results have primarily focused on LLM adaptation. However, **T2L** can be directly applied to other LLMs or to adapt vision language models. Finally, the potential for **T2L** trained on a smaller base model to transfer effectively to larger models within the same architecture class remains an open area for exploration.

Limitations. We only consider LoRA as the output space of the hypernetwork. We believe there are more efficient ways to modulate LLMs given a text description, e.g., directly modulating the activations of the base model. Also, we believe the compression achieved by **T2L** can be further optimized using well-designed inductive biases. Finally, although **T2L** exhibits robustness and signs of scalability, it still does not reach the benchmark performance of task-specific LoRAs in a zero-shot manner. Achieving such potent zero-shot adaption is still one of the biggest challenges for **T2L**.

Acknowledgment

We thank David Ha for suggesting “Text-to-LoRA” as the title of the paper. We thank anonymous reviewers for their constructive feedback, which we incorporate to improve the quality of the paper.

Impact Statement

This paper introduces Text-to-LoRA (T2L), a novel approach that significantly lowers the barrier to adapting large foundation models for specific tasks. Traditionally, customizing models like LLMs requires resource-intensive fine-tuning on specific datasets for each new application, limiting accessibility and slowing down deployment. T2L overcomes this by training a hypernetwork to generate task-specific Low-Rank Adapters (LoRAs) instantly, using only a natural language description of the target task as input. This eliminates the need for per-task fine-tuning datasets and lengthy optimization processes, enabling rapid, on-the-fly adaptation with minimal computational overhead during inference, thereby making powerful model customization more accessible.

The broader impact of T2L lies in its potential to democratize the specialization of powerful AI systems by enabling adaptation through intuitive text instructions. While T2L demonstrates effective compression and promising zero-shot generalization to unseen tasks similar to those encountered during training, potential pitfalls exist that warrant consideration. Its performance is notably sensitive to the quality and clarity of the natural language task descriptions; poorly phrased or misaligned instructions could lead to suboptimal or incorrect adaptations, potentially hindering reliability in real-world user scenarios. Furthermore, while T2L significantly advances instant adaptation, its generalization capability to task types fundamentally different from its training distribution (e.g., beyond the SNI-derived benchmarks) needs further investigation, and it may not yet fully match the performance ceiling of adapters meticulously fine-tuned on extensive, high-quality datasets for highly complex or specialized domains.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Beck, J., Jackson, M. T., Vuorio, R., and Whiteson, S. Hypernetworks in meta-reinforcement learning. In *Conference on Robot Learning*, pp. 1478–1487. PMLR, 2023.
- BehnamGhader, P., Adlakha, V., Mosbach, M., Bahdanau, D., Chapados, N., and Reddy, S. LLM2Vec: Large language models are secretly powerful text encoders. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=IW1PR7vEBf>.
- Bisk, Y., Zellers, R., Bras, R. L., Gao, J., and Choi, Y. Piqa: Reasoning about physical commonsense in natural language. In *Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- Brüel-Gabrielsson, R., Zhu, J., Bhardwaj, O., Choshen, L., Greenewald, K., Yurochkin, M., and Solomon, J. Compress then serve: Serving thousands of lora adapters with little overhead. *arXiv preprint arXiv:2407.00066*, 2024.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code, 2021.
- Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019.

- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Deb, B., Awadallah, A. H., and Zheng, G. Boosting natural language generation from instructions with meta-learning. In Goldberg, Y., Kozareva, Z., and Zhang, Y. (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 6792–6808, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.456. URL <https://aclanthology.org/2022.emnlp-main.456>.
- Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332, 2022.
- Digre, K. B. and Brennan, K. Shedding light on photophobia. *Journal of Neuro-ophthalmology*, 32(1):68–81, 2012.
- Dong, Q., Li, L., Dai, D., Zheng, C., Ma, J., Li, R., Xia, H., Xu, J., Wu, Z., Chang, B., et al. A survey on in-context learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 1107–1128, 2024.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., and Smith, N. A. Don’t stop pretraining: Adapt language models to domains and tasks. *arXiv preprint arXiv:2004.10964*, 2020.
- Ha, D., Dai, A., and Le, Q. V. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016.
- He, Y., Zheng, S., Tay, Y., Gupta, J., Du, Y., Aribandi, V., Zhao, Z., Li, Y., Chen, Z., Metzler, D., et al. Hyperprompt: Prompt-based task-conditioning of transformers. In *International conference on machine learning*, pp. 8678–8690. PMLR, 2022.
- Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Iverson, H. and Peters, M. E. Hyperdecoders: Instance-specific decoders for multi-task nlp. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 1715–1730, 2022.
- Iverson, H., Bhagia, A., Wang, Y., Hajishirzi, H., and Peters, M. E. Hint: Hypernetwork instruction tuning for efficient zero-and few-shot generalisation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11272–11288, 2023.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Keisuke, S., Ronan, L. B., Chandra, B., and Yejin, C. Winogrande: An adversarial winograd schema challenge at scale. 2019.
- Kim, H., Sasaki, S., Hoshino, S., and Honda, U. A single linear layer yields task-adapted low-rank matrices. *arXiv preprint arXiv:2403.14946*, 2024.
- Kopieczko, D. J., Blankevoort, T., and Asano, Y. M. VeRA: Vector-based random matrix adaptation. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NjNfLdxr3A>.
- Korthikanti, V. A., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoenybi, M., and Catanzaro, B. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5:341–353, 2023.
- Lewis, M. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*, 2019.
- Li, Z., Zhang, X., Zhang, Y., Long, D., Xie, P., and Zhang, M. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*, 2023.
- Liu, J., Xia, C. S., Wang, Y., and Zhang, L. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=1qv610Cu7>.
- Lv, C., Li, L., Zhang, S., Chen, G., Qi, F., Zhang, N., and Zheng, H.-T. Hyperlora: Efficient cross-task generalization via constrained low-rank adapters generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 16376–16393, 2024.

- Mahabadi, R. K., Ruder, S., Dehghani, M., and Henderson, J. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks. *arXiv preprint arXiv:2106.04489*, 2021.
- Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*, 2018.
- Mu, J., Li, X., and Goodman, N. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36, 2024.
- Ortiz-Barajas, J.-G., Gomez-Adorno, H., and Solorio, T. Hyperloader: Integrating hypernetwork-based lora and adapter layers into multi-task transformers for sequence labelling. *arXiv preprint arXiv:2407.01411*, 2024.
- Ostapenko, O., Su, Z., Ponti, E. M., Charlin, L., Roux, N. L., Pereira, M., Caccia, L., and Sordani, A. Towards modular llms by building and reusing a library of lorae. *arXiv preprint arXiv:2405.11157*, 2024.
- Phang, J., Mao, Y., He, P., and Chen, W. Hypertuning: Toward adapting large language models without back-propagation. In *International Conference on Machine Learning*, pp. 27854–27875. PMLR, 2023.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21 (140):1–67, 2020.
- Schmidhuber, J. Discovering neural nets with low kolmogorov complexity and high generalization capability. *Neural Networks*, 10(5):857–873, 1997.
- Schug, S., Kobayashi, S., Akram, Y., Sacramento, J., and Pascanu, R. Attention as a hypernetwork. *arXiv preprint arXiv:2406.05816*, 2024.
- Stanley, K. O. and Miikkulainen, R. A taxonomy for artificial embryogeny. *Artificial life*, 9(2):93–130, 2003.
- Tay, Y., Dehghani, M., Rao, J., Fedus, W., Abnar, S., Chung, H. W., Narang, S., Yogatama, D., Vaswani, A., and Metzler, D. Scale efficiently: Insights from pre-training and fine-tuning transformers. *arXiv preprint arXiv:2109.10686*, 2021.
- Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- Von Oswald, J., Henning, C., Grewe, B. F., and Sacramento, J. Continual learning with hypernetworks. *arXiv preprint arXiv:1906.00695*, 2019.
- Wang, Y., Mishra, S., Alipoormolabashi, P., Kordi, Y., Mirzaei, A., Naik, A., Ashok, A., Dhanasekaran, A. S., Arunkumar, A., Stap, D., et al. Super-naturalinstructions: Generalization via declarative instructions on 1600+ nlp tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 5085–5109, 2022.
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.
- Wurtz, R. H., Joiner, W. M., and Berman, R. A. Neuronal mechanisms for visual stability: progress and problems. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 366(1564):492–503, 2011.
- Xiao, Z., Held, W., Liu, Y., and Yang, D. Task-agnostic low-rank adapters for unseen English dialects. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 7857–7870, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.487. URL <https://aclanthology.org/2023.emnlp-main.487>.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- Zhang, C., Ren, M., and Urtasun, R. Graph hypernetworks for neural architecture search. *arXiv preprint arXiv:1810.05749*, 2018.
- Zhang, X., Zhang, Y., Long, D., Xie, W., Dai, Z., Tang, J., Lin, H., Yang, B., Xie, P., Huang, F., et al. mgte: Generalized long-context text representation and reranking models for multilingual text retrieval. *arXiv preprint arXiv:2407.19669*, 2024.
- Zhao, J., Wang, T., Abid, W., Angus, G., Garg, A., Kinnison, J., Sherstinsky, A., Molino, P., Addair, T., and Rishi, D. Lora land: 310 fine-tuned llms that rival gpt-4, a technical report. *arXiv preprint arXiv:2405.00732*, 2024.

A. Generalization to Llama and Gemma Models

Table 7: Zero-shot performance with Llama-3.1-8B-Instruct as the base language model.

	ArcC (acc)	ArcE (acc)	BQ (acc)	GSM8K (acc)	HS (acc)	OQA (acc)	PIQA (acc)	WG (acc)	HE (pass@1)	MBPP (pass@1)	Avg.
Llama-3.1-8B-Instruct	73.3	90.6	80.4	75.7	66.6	75.4	79.8	55.3	66.5	68.7	73.2
3-shot ICL	80.7	91.9	80.0	75.7	59.3	77.6	80.9	61.3	66.5	70.4	74.4
Prepending task desc.	80.2	92.5	79.9	75.7	69.8	78.4	81.7	62.4	68.3	70.2	75.9
Multi-task LoRA	82.0	92.8	83.3	77.6	70.8	81.8	83.8	60.3	63.4	69.4	76.5
T2L (SFT) L	82.4	92.9	84.4	79.1	72.8	81.8	81.2	60.0	64.6	69.9	76.9

Table 8: Zero-shot performance with Gemma-2-2B-Instruct as the base language model.

	ArcC (acc)	ArcE (acc)	BQ (acc)	GSM8K (acc)	HS (acc)	OQA (acc)	PIQA (acc)	WG (acc)	HE (pass@1)	MBPP (pass@1)	Avg.
Gemma-2-2B-Instruct	73.7	89.9	81.0	55.6	55.2	71.0	71.0	53.8	43.9	12.3	60.7
3-shot ICL	72.4	88.9	82.5	55.6	55.7	72.6	67.6	53.7	43.9	43.1	63.6
Prepending task desc. w/ ICL	72.4	88.9	82.5	55.6	55.7	72.6	67.6	53.7	43.9	43.1	63.6
Multi-task LoRA w/ ICL	73.5	89.4	81.6	57.2	59.5	74.6	69.4	58.1	39.0	50.4	65.2
T2L (SFT) L w/ ICL	74.0	89.8	81.8	55.1	62.5	73.9	75.2	58.7	41.5	51.5	66.4

In this section, we explore the generality of our proposed architectures to different model families and sizes. Tables 7 and 8 show the benchmark performance of **T2L L** compared to various baselines using Llama-3.1-8B-Instruct and Gemma-2-2B-Instruct as the base models, respectively. With Gemma base model, we utilize ICL for all approaches as it drastically improves the performance on the MBPP benchmark. We see that **T2L** consistently outperforms the baselines across all tested models with varying model sizes and architectures. We note that **T2L** are trained with the same set of hyperparameters across base models.

B. Training Description Sources

Table 9: Performance of SFT-trained **T2L** with two different training description sources.

	ArcC (acc)	ArcE (acc)	BQ (acc)	GSM8K (acc)	HS (acc)	OQA (acc)	PIQA (acc)	WG (acc)	HE (pass@1)	MBPP (pass@1)	Avg.
T2L (SFT) L	77.5	88.9	85.0	45.8	66.5	75.5	82.1	64.2	39.2	51.9	67.7
T2L (SFT) L w/ SNI def.	75.3	87.4	85.0	45.9	63.6	73.5	80.9	61.8	38.2	53.8	66.5

In this experiment, we explore the impact of the sources of the training task descriptions: SNI and chatGPT (Appendix L). Table 9 shows that using task definitions provided by the SNI datasets reduces the zero-shot benchmark performance of **T2L**. As the SNI datasets are crowd-sourced, we hypothesized that the task descriptions might have inconsistent template or varied levels of details. Thus, it is harder for **T2L** to learn and generalize.

C. Scaling the Number of Training Tasks with Fixed Compute

We study the impact of the number of training tasks on the zero-shot benchmark performance of **T2L** in the SFT setting, where all **T2L** instances are trained for roughly the same number of gradient steps (see details in Appendix H). Overall, we find that increasing the number of training tasks improves the average zero-shot benchmark performance of the hypernetwork (Figure 1 and Table 10). This result hints at the plausible scalability of **T2L** and positive transfer between tasks.

Table 10: Benchmark performance of SFT-trained T2L with varying numbers of training tasks. We show results with {64, 128, 256, 479} tasks. $\blacktriangle$ ($\blacktriangledown$) indicates increased (decreased) performance compared to the previous increment in the number of training tasks and training budget.

	Number of tasks	ArcC (acc)	ArcE (acc)	BQ (acc)	GSM8K (acc)	HS (acc)	OQA (acc)	PIQA (acc)	WG (acc)	HE (pass@1)	MBPP (pass@1)	Avg.
T2L (SFT) L	479	77.2	89.0	85.0	46.3	66.5	73.6	82.6	61.8	39.2	44.3	66.6 $\blacktriangledown$
	256	76.6	89.1	84.8	47.0	67.7	73.5	82.8	62.4	39.6	51.0	67.5 $\blacktriangle$
	128	76.2	89.0	85.3	46.2	67.9	71.7	82.6	59.9	40.5	51.3	67.0 $\blacktriangle$
	64	75.5	88.0	84.5	43.9	65.5	70.7	80.5	59.5	39.8	51.7	66.0
T2L (SFT) M	479	77.5	89.0	85.0	45.8	66.5	71.9	82.1	61.4	41.3	50.1	67.1 $\blacktriangle$
	256	76.1	88.2	85.3	45.4	65.6	72.7	81.7	62.3	36.8	50.6	66.5 $\blacktriangle$
	128	75.5	87.8	85.3	46.1	66.6	71.6	81.7	62.2	39.8	44.9	66.1 $\blacktriangle$
	64	73.6	87.7	84.5	43.2	64.6	70.5	79.9	56.0	40.7	51.4	65.2
T2L (SFT) S	479	75.8	88.5	83.9	45.6	64.2	71.9	82.3	61.5	36.2	45.0	65.5 $\blacktriangle$
	256	76.1	88.4	83.0	47.3	65.0	71.7	82.5	58.1	36.2	39.1	64.8 $\blacktriangle$
	128	75.6	87.7	84.9	46.5	65.7	72.7	81.0	59.6	39.0	28.1	64.1 $\blacktriangledown$
	64	75.4	88.4	85.0	43.1	64.8	70.7	81.5	51.6	39.4	46.7	64.7

D. LoRAs of Similar Tasks

Here, we investigate the relationship between LoRA adapters by inspecting their similarity in the parameter space, performance on the benchmarks, and similarity of their description embeddings. To measure adapter similarity, we compute the cosine similarity of the concatenation of flattened low-rank A and B matrices of all layers. In the top row of Figure 6, we plot the adapters’ similarity against task description similarity (using the mean embedding of each task). We find no correlation between the cosine similarity of the adapters’ weights (y-axis) and the task embedding similarity (x-axis) indicated by near-zero Pearson correlation coefficients.

In the bottom row of Figure 6, we change the y-axis from adapters’ relative benchmark performance to benchmark-specific adapters. We find a positive correlation between the relative benchmark performance of SNI-trained adapters and the task embedding similarity. That is, adapters perform better on a benchmark if their task descriptions are similar to those of the benchmark. However, despite their similar functionalities, adapters with similar descriptions are not similar in the parameter space. We believe that this relationship has a significant impact on the limited generalization of reconstruction-trained T2L. We further discuss this topic in Appendix K.

E. Hyperparameter Settings

Table 11: Hyperparameters for training a task-specific LoRA adapter.

Hyperparameters	Task-specific LoRA	T2L (SFT)	T2L (recon)
Batch size	8	8	Number of the target LoRAs
Gradient accumulation steps	1	1	1
Max learning rate	8×10^{-5}	2.5×10^{-5}	10^{-3}
Max gradient norm	1.0	1.0	1.0
NEFTune noise alpha	5.0	5.0	5.0
Warmup fraction	0.1	0.1	0.1
Learning rate scheduler	Linear with warm up	Linear with warm up	Linear with warm up

Table 11 and Listing 1 show the training configuration of all models trained in this work. For LoRA reconstruction training, each prediction target is an entirety of a LoRA adapter. That is, there is a total of 479 training samples for 479 SNI tasks. Thus, we increase the epochs to 100,000 to ensure that T2L converges.

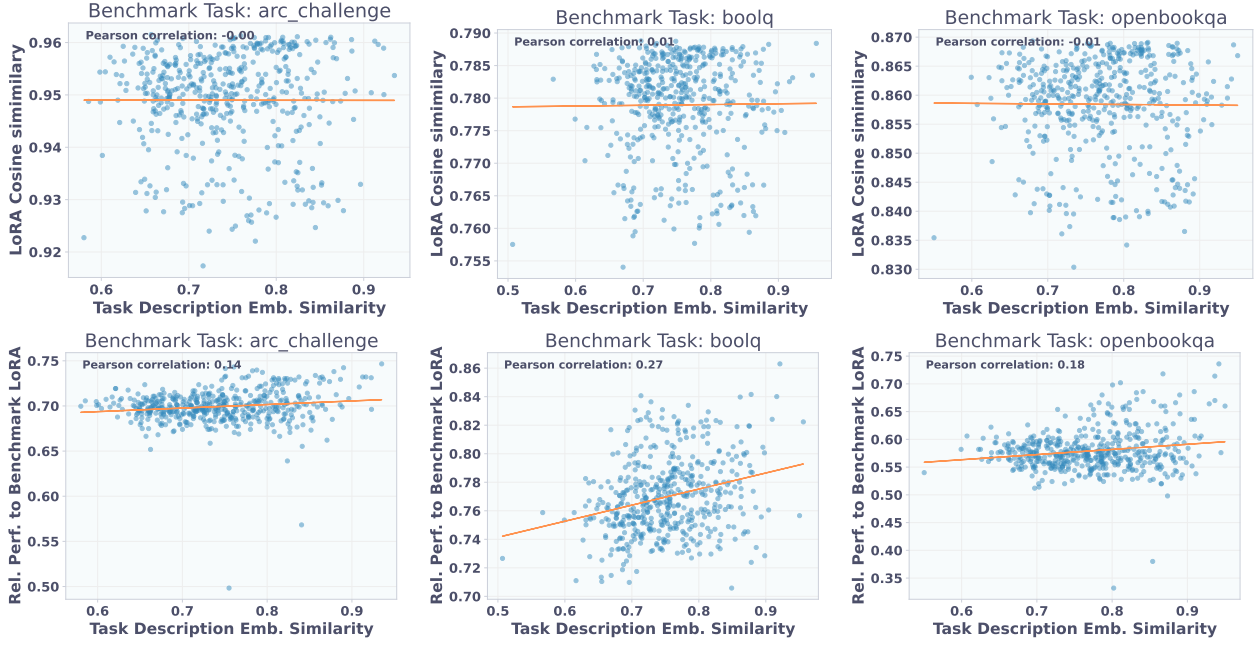


Figure 6: **Top row:** Each plot shows the similarity between a benchmark LoRA adapter and 479 SNI-trained adapters in the weight space (y-axis) against their similarity in the task embedding space (x-axis). **Bottom row:** Each plot shows SNI-trained adapters’ performance relative to a benchmark adapter (y-axis) with the same x-axis. We can see that LoRAs with similar description embeddings to the benchmarks’ perform better in those benchmarks, suggesting their shared functionalities. However, LoRAs with similar functionalities are not nearby in the parameter space.

F. Additional Details of T2L Architectures

Listings 2 and 3 show the details of the backbone of T2L. Specifically, the size of the module and layer embedding ($E[m]$ and $E[l]$) is 32D. Together, they form a dictionary of 34 learnable embeddings (32 layers + 2 target modules). The task encoder is a linear layer that takes in a text embedding (1024D for the `gte` embedding and 4096D for `Mistral` embedding) and outputs a 64D vector. The encoded task, module, and layer embedding are concatenated and then fed into `mlp0` followed by a residual MLP block `mlp1`. At this point, for **M** and **S**, we add a 128D A/B embedding to the residual stream. The output is then fed to another residual MLP block `mlp2`. At this point, for **S**, we add a 128D rank embedding to the residual stream. After this, we feed the activation to the last MLP block. The output of the last MLP block is then fed to a linear head, whose output size is as follows:

- **L**: $2 \times r \times d$ giving both A and B matrices
- **M**: $r \times d$ giving a low-rank matrix A or B depending on the A/B embedding
- **S**: d giving a rank of a low-rank matrix depending on both the A/B embedding and the rank embedding.

For ease of explanation, we assume d is the same for the input and the output space of a linear transformation. In practice, $d_{\text{in}} = d_{\text{out}} = 4096$ for `q_proj` module and $d_{\text{in}} = 4096, d_{\text{out}} = 1024$ for `v_proj` module. $r = 8$ for all adapters in this work. Finally, we list the number of trainable parameters of each architecture: 55, 252, 992 for **L**, 34, 282, 240 for **M**, 4, 923, 392 for **S**, 3, 407, 872 for LoRA.

G. T2L Initialization

We use *Bias-HyperInit* (Beck et al., 2023) to initialize **L** T2L. Bias-HyperInit initializes the linear output head of the hypernetwork such that the weights are all zeros and the bias matches the initialization of the underlying layers. In our work, this corresponds to the output bias of the **L** hypernetwork being initialized to $U(-\frac{1}{d}, \frac{1}{d})$ for the A head and all zero for the B head to match the initialization of traditional LoRA. For other architectures, we aim to match the gradient magnitude to

```
{
  "alpha_pattern": {},
  "auto_mapping": null,
  "base_model_name_or_path": "models/Mistral-7B-Instruct-v0.2",
  "bias": "none",
  "fan_in_fan_out": false,
  "inference_mode": true,
  "init_lora_weights": true,
  "layer_replication": null,
  "layers_pattern": null,
  "layers_to_transform": null,
  "loftq_config": {},
  "lora_alpha": 16,
  "lora_dropout": 0.05,
  "megatron_config": null,
  "megatron_core": "megatron.core",
  "modules_to_save": null,
  "peft_type": "LORA",
  "r": 8,
  "rank_pattern": {},
  "revision": null,
  "target_modules": [
    "q_proj",
    "v_proj"
  ],
  "task_type": "CAUSAL_LM",
  "use_dora": false,
  "use_rslora": true
}
```

Listing 1: The parameter-efficient fine-tuning (PEFT) config for all LoRA adapters.

L at the beginning of training. That is, for **M** architecture, we initialize the bias of the output head to $U(-\frac{1}{\sqrt{2d}}, \frac{1}{\sqrt{2d}})$. Finally, **S** output bias is initialized to $U(-\frac{1}{\sqrt{r2d}}, \frac{1}{\sqrt{r2d}})$. Without this explicit hypernetwork initialization, the training is unstable, and often leads to failed training runs.

H. Training Details

All models trained in this work fit in a single H100 GPU (80GB of VRAM). Notably, SFT requires much more memory because of the need to backpropagate the gradient through the base LLM. Reconstruction training, on the other hand, should be possible in a modern consumer-grade GPU.

For reconstruction training, we fix the training epochs to be 100K but scale the batch size to match the number of target LoRA adapters. This means the model trains much faster for a lower number of target LoRAs while maintaining the same number of optimizer steps. For reference, training to reconstruct 9 benchmark-specific LoRAs takes around 10 minutes to complete, while training to reconstruct 479 SNI LoRA adapters takes around 10 hours.

For SFT training with fixed compute budget, we aim to keep the number of optimizer steps the same as we do for reconstruction training. However, since we cannot fit all fine-tuning samples, we scale the number of epochs inverse to the number of training tasks.

Additionally, for reconstruction training, instead of predicting the weights directly, **T2L** learns to predict the z-score of a normal distribution of each weight entry in the low-rank **A**, **B** matrices. At test time, the output is multiplied by the standard

deviation of each element before adding to the mean, converting the prediction to the correct scale.

I. Ad-hoc FLOPs Analysis

Let S be the sequence length, H be the hidden size, and L be the number of layers of a Transformer-based LLM. We use the following equations for computing the matrix multiplications (GEMMs) FLOPs (Korthikanti et al., 2023).

FLOPs for Self-Attention (per layer): $8 \times S \times H^2 + 4 \times H \times S^2$

FLOPs for FFN (per layer): $16 \times S \times H^2$

Per Transformer Block Total FLOPs: $24 \times S \times H^2 + 4 \times H \times S^2$

Setup for comparison:

- 3-shot ICL examples are approximately 256 tokens long
- Question instances are approximately 64 tokens long
- Task descriptions are approximately 48 tokens long
- We consider one question instance as the main input to the base model
- We only consider input tokens for the FLOPs calculation
- We use ‘Mistral-7B-Instruct-v0.2’ as the base model ($S = 256 + 64$ (3-shot ICL + question instance), $H = 4096$, $L = 32$)
- When the based model is used with T2L, we do not include 3-shot ICL ($S = 64$ (question instance), $H = 4096$, $L = 32$)
- We use ‘gte-large-en-v1.5’ as the task description encoder ($S = 48$ (task description), $H = 1024$, $L = 24$)
- We use the M hypernetwork architecture detailed in the Appendix F

I.1. T2L per instance FLOPs

gte-large-en-v1.5: FLOPs = $24 \times (24 \times 48 \times 1024^2 + 4 \times 1024 \times 48^2) = 0.029$ TFLOPs/instance

Hypernetwork (M): FLOPs = $2 \times 1024 \times 64 + 4 \times 4 \times 128 \times 512 + 128 \times 4096 \times 8 = 0.000005$ TFLOPs/instance

Base LLM w/o ICL: FLOPs = $32 \times (24 \times 64 \times 4096^2 + 4 \times 4096 \times 64^2) = 0.827$ TFLOPs/instance

Total FLOPs = $0.029 + 0.000005 + 0.827 = \mathbf{0.856005}$ TFLOPs/instance

I.2. Base LLM with 3-shot ICL

Total FLOPs = $32 \times (24 \times (256 + 64) \times 4096^2 + 4 \times (4096) \times (256 + 64)^2) = \mathbf{4.177}$ TFLOPs/instance

Based on this calculation, we can see that the adaptation cost of T2L is significantly cheaper than 3-shot ICL—more than 4x FLOPs reduction, saving compute within the first question instance.

J. Training and Evaluation Datasets

We use 500 SNI datasets publicly available at <https://huggingface.co/Lots-of-LoRAs>. 479 tasks are used for training and the rest for evaluation. Specifically, we use the following evaluation tasks: task_035, task_039, task_1557, task_202, task_304, task_362, task_614, task_701, task_706, task_710, task_726. For the in-context learning baseline, we use 3-shot in-context examples taken from the training split of each benchmark except MBPP that has an explicit split for in-context prompting. HumanEval only has the test split, therefore it is always evaluated against in the zero-shot manner.

Training Tasks

"Lots-of-LoRAs/task742_lhoestq_answer_generation_frequency",
 "Lots-of-LoRAs/task1198_atomic_classification_owant",
 "Lots-of-LoRAs/task717_mmmlu_answer_generation_logical_fallacies",
 "Lots-of-LoRAs/task705_mmmlu_answer_generation_high_school_macro_economics",
 "Lots-of-LoRAs/task775_entinno_paraphrase_generation",
 "Lots-of-LoRAs/task636_extract_and_sort_unique_alphabets_in_a_list",
 "Lots-of-LoRAs/task084_babi_t1_single_supporting_fact_identify_relevant_fact",
 "Lots-of-LoRAs/task1711_poki_text_generation",
 "Lots-of-LoRAs/task140_detoxifying-lms_classification_style",
 "Lots-of-LoRAs/task1448_disease_entity_extraction_ncbi_dataset",
 "Lots-of-LoRAs/task453_swag_answer_generation",
 "Lots-of-LoRAs/task1207_atomic_classification_allocation",
 "Lots-of-LoRAs/task734_mmmlu_answer_generation_sociology",
 "Lots-of-LoRAs/task298_storycloze_correct_end_classification",
 "Lots-of-LoRAs/task587_amazonfood_polarity_correction_classification",
 "Lots-of-LoRAs/task703_mmmlu_answer_generation_high_school_geography",
 "Lots-of-LoRAs/task147_afs_argument_similarity_gay_marriage",
 "Lots-of-LoRAs/task564_discofuse_classification",
 "Lots-of-LoRAs/task1341_msr_text_classification",
 "Lots-of-LoRAs/task201_mnil_neutral_classification",
 "Lots-of-LoRAs/task890_gowd_classification",
 "Lots-of-LoRAs/task908_dialogre_identify_familial_relationships",
 "Lots-of-LoRAs/task1428_country_surface_area",
 "Lots-of-LoRAs/task325_jigsaw_classification_identity_attack",
 "Lots-of-LoRAs/task1669_md_gender_bias_text_modification",
 "Lots-of-LoRAs/task246_dream_question_generation",
 "Lots-of-LoRAs/task357_casino_classification_negotiation_small_talk",
 "Lots-of-LoRAs/task1518_limit_answer_generation",
 "Lots-of-LoRAs/task1148_maximum_ascii_value",
 "Lots-of-LoRAs/task1605_ethos_text_classification",
 "Lots-of-LoRAs/task867_mawps_multiop_question_answering",
 "Lots-of-LoRAs/task209_stancedetection_classification",
 "Lots-of-LoRAs/task751_svamp_subtraction_question_answering",
 "Lots-of-LoRAs/task161_count_words_containing_letter",
 "Lots-of-LoRAs/task105_story_cloze_rocstories_sentence_generation",
 "Lots-of-LoRAs/task645_summarization",
 "Lots-of-LoRAs/task442_com_qa_paraphrase_question_generation",
 "Lots-of-LoRAs/task075_squad1.1_answer_generation",
 "Lots-of-LoRAs/task269_csrg_counterfactual_story_generation",
 "Lots-of-LoRAs/task1568_propara_classification",
 "Lots-of-LoRAs/task834_mathdataset_classification",
 "Lots-of-LoRAs/task1603_orcaslow_sentence_generation",
 "Lots-of-LoRAs/task685_mmmlu_answer_generation_clinical_knowledge",
 "Lots-of-LoRAs/task083_babi_t1_single_supporting_fact_answer_generation",
 "Lots-of-LoRAs/task390_torque_text_span_selection",
 "Lots-of-LoRAs/task750_aqua_multiple_choice_answering",
 "Lots-of-LoRAs/task1631_openpi_answer_generation",
 "Lots-of-LoRAs/task1529_scital1.1_classification",
 "Lots-of-LoRAs/task749_yelp_restaurant_review_classification",
 "Lots-of-LoRAs/task1217_atomic_answer_generation",
 "Lots-of-LoRAs/task725_mmmlu_answer_generation_nutrition",
 "Lots-of-LoRAs/task889_goemotions_classification",
 "Lots-of-LoRAs/task492_mwsc_incorrect_answer_generation",
 "Lots-of-LoRAs/task620_ohsumed_medical_subject_headings_answer_generation",
 "Lots-of-LoRAs/task084_all_elements_commonsense_motiv_text_generation",
 "Lots-of-LoRAs/task641_esnl_classification",
 "Lots-of-LoRAs/task318_stereoset_classification_gender",
 "Lots-of-LoRAs/task846_pubmedqa_classification",
 "Lots-of-LoRAs/task316_crows-pairs_classification_stereotype",
 "Lots-of-LoRAs/task1188_count_max_freq_char",
 "Lots-of-LoRAs/task629_dbpedia_14_classification",
 "Lots-of-LoRAs/task770_pawssx_english_text_modification",
 "Lots-of-LoRAs/task1482_gene_extraction_chempot_dataset",
 "Lots-of-LoRAs/task499_extract_and_add_all_numbers_from_list",
 "Lots-of-LoRAs/task955_wiki_auto_style_transfer",
 "Lots-of-LoRAs/task719_mmmlu_answer_generation_management",
 "Lots-of-LoRAs/task723_mmmlu_answer_generation_moral_disputes",
 "Lots-of-LoRAs/task087_new_operator_addsub_arithmetic",
 "Lots-of-LoRAs/task211_logic2text_classification",
 "Lots-of-LoRAs/task901_freebase_qa_category_question_generation",
 "Lots-of-LoRAs/task1483_chemical_extraction_chempot_dataset",
 "Lots-of-LoRAs/task089_swap_words_verification",
 "Lots-of-LoRAs/task627_xlwc_word_with_same_meaning_sentence_generation",
 "Lots-of-LoRAs/task153_tomqa_find_location_hard_clean",
 "Lots-of-LoRAs/task1342_amazon_us_reviews_title",
 "Lots-of-LoRAs/task828_copa_commonsense_cause_effect",
 "Lots-of-LoRAs/task064_all_elements_except_first_l",
 "Lots-of-LoRAs/task1387_anli_r3_entailment",
 "Lots-of-LoRAs/task400_paws_paraphrase_classification",
 "Lots-of-LoRAs/task1294_wiki_qa_answer_verification",
 "Lots-of-LoRAs/task243_count_elements_in_set_intersection",
 "Lots-of-LoRAs/task1572_samsun_summary",
 "Lots-of-LoRAs/task151_swap_max_min",
 "Lots-of-LoRAs/task574_air_dialogue_sentence_generation",
 "Lots-of-LoRAs/task428_senteval_inversion",
 "Lots-of-LoRAs/task366_synthetic_return_primes",
 "Lots-of-LoRAs/task926_coached_conv_pref_word_generation",
 "Lots-of-LoRAs/task1503_hatexplain_classification",
 "Lots-of-LoRAs/task130_scan_structured_text_generation_command_action_long",
 "Lots-of-LoRAs/task515_senteval_odd_word_out",
 "Lots-of-LoRAs/task151_tomqa_find_location_easy_clean",
 "Lots-of-LoRAs/task084_ohsumed_abstract_title_generation",
 "Lots-of-LoRAs/task1562_zest_text_modification",
 "Lots-of-LoRAs/task632_dbpedia_14_classification",
 "Lots-of-LoRAs/task966_ruletaur_fact_checking_based_on_given_context",
 "Lots-of-LoRAs/task605_find_the_longest_common_subsequence_in_two_lists",
 "Lots-of-LoRAs/task1487_organism_substance_extraction_anem_dataset",
 "Lots-of-LoRAs/task707_mmmlu_answer_generation_high_school_microeconomics",
 "Lots-of-LoRAs/task1378_quarel_incorrect_answer_generation",
 "Lots-of-LoRAs/task1489_sarcasmdetection_tweet_classification",

Training Tasks (cont.)

"Lots-of-LoRAs/task1567_propara_question_generation",
 "Lots-of-LoRAs/task1584_deal_or_no_dialog_classification",
 "Lots-of-LoRAs/task1494_drac_conversion",
 "Lots-of-LoRAs/task691_mmmlu_answer_generation_college_physics",
 "Lots-of-LoRAs/task728_mmmlu_answer_generation_professional_accounting",
 "Lots-of-LoRAs/task219_rocstories_title_answer_generation",
 "Lots-of-LoRAs/task964_librispeech_asr_text_auto_completion",
 "Lots-of-LoRAs/task1509_evaluation_antonyms",
 "Lots-of-LoRAs/task582_naturalquestion_answer_generation",
 "Lots-of-LoRAs/task455_swag_context_text_generation",
 "Lots-of-LoRAs/task063_librispeech_asr_next_word_prediction",
 "Lots-of-LoRAs/task382_hybridqa_answer_generation",
 "Lots-of-LoRAs/task859_prost_question_generation",
 "Lots-of-LoRAs/task1393_superglue_copa_text_completion",
 "Lots-of-LoRAs/task1565_triviaqa_classification",
 "Lots-of-LoRAs/task1720_civil_comments_toxicity_classification",
 "Lots-of-LoRAs/task670_ambigua_question_generation",
 "Lots-of-LoRAs/task689_mmmlu_answer_generation_college_mathematics",
 "Lots-of-LoRAs/task324_jigsaw_classification_disagree",
 "Lots-of-LoRAs/task1420_mathqa_general",
 "Lots-of-LoRAs/task618_amazonreview_summary_text_generation",
 "Lots-of-LoRAs/task625_xlwc_true_or_false_answer_generation",
 "Lots-of-LoRAs/task377_remove_words_of_given_length",
 "Lots-of-LoRAs/task929_products_reviews_classification",
 "Lots-of-LoRAs/task286_storycloze_correct_end_classification",
 "Lots-of-LoRAs/task852_synthetic_multiply_odds",
 "Lots-of-LoRAs/task1332_check_leap_year",
 "Lots-of-LoRAs/task1444_round_power_of_two",
 "Lots-of-LoRAs/task850_synthetic_longest_palindrome",
 "Lots-of-LoRAs/task708_mmmlu_answer_generation_high_school_physics",
 "Lots-of-LoRAs/task1292_yelp_review_full_text_categorization",
 "Lots-of-LoRAs/task110_logic2text_sentence_generation",
 "Lots-of-LoRAs/task155_count_nouns_verbs",
 "Lots-of-LoRAs/task429_senteval_tense",
 "Lots-of-LoRAs/task245_check_presence_in_set_intersection",
 "Lots-of-LoRAs/task137_detoxifying-lms_classification_toxicity",
 "Lots-of-LoRAs/task1566_propara_structured_text_generation",
 "Lots-of-LoRAs/task1148_country_capital",
 "Lots-of-LoRAs/task624_event2mind_word_generation",
 "Lots-of-LoRAs/task022_cosmosqa_passage_inappropriate_binary",
 "Lots-of-LoRAs/task118_semeval_open_vocabulary_mathematical_answer_generation",
 "Lots-of-LoRAs/task687_mmmlu_answer_generation_college_chemistry",
 "Lots-of-LoRAs/task1167_penn_treebank_coarse_pos_tagging",
 "Lots-of-LoRAs/task080_boolq_yes_no_question",
 "Lots-of-LoRAs/task092_winegrade_answer_generation",
 "Lots-of-LoRAs/task1502_hatexplain_classification",
 "Lots-of-LoRAs/task805_mawps_addsub_question_answering",
 "Lots-of-LoRAs/task181_outcome_extraction",
 "Lots-of-LoRAs/task020_pre-answer-generation_easy",
 "Lots-of-LoRAs/task698_mmmlu_answer_generation_global_facts",
 "Lots-of-LoRAs/task956_leetcode_420_strong_password_check",
 "Lots-of-LoRAs/task732_mmmlu_answer_generation_public_relations",
 "Lots-of-LoRAs/task721_medical_generation_medical_genetics",
 "Lots-of-LoRAs/task370_synthetic_remove_divisible_by_3",
 "Lots-of-LoRAs/task1400_obqa_incorrect_answer_generation",
 "Lots-of-LoRAs/task1199_atomic_classification_xatr",
 "Lots-of-LoRAs/task1606_ethos_text_classification",
 "Lots-of-LoRAs/task288_gigaword_summarization",
 "Lots-of-LoRAs/task1670_md_gender_bias_text_modification",
 "Lots-of-LoRAs/task207_max_element_lists",
 "Lots-of-LoRAs/task1208_atomic_classification_isbefore",
 "Lots-of-LoRAs/task457_matres_conditional_classification",
 "Lots-of-LoRAs/task1308_amazonreview_category_classification",
 "Lots-of-LoRAs/task1310_amazonreview_rating_classification",
 "Lots-of-LoRAs/task874_opus_xhosanavy_sr",
 "Lots-of-LoRAs/task1541_agnews_classification",
 "Lots-of-LoRAs/task1609_xquad_en_question_generation",
 "Lots-of-LoRAs/task1210_logic2text_structured_text_generation",
 "Lots-of-LoRAs/task1318_country_national_dish",
 "Lots-of-LoRAs/task365_synthetic_remove_vowels",
 "Lots-of-LoRAs/task755_find_longest_substring_and_replace_its_sorted_lowercase",
 "Lots-of-LoRAs/task123_conala_sort_dictionary",
 "Lots-of-LoRAs/task1316_remove_duplicates_string",
 "Lots-of-LoRAs/task1378_quarel_correct_answer_generation",
 "Lots-of-LoRAs/task475_yelp_polarity_classification",
 "Lots-of-LoRAs/task903_deceptive_opinion_spam_classification",
 "Lots-of-LoRAs/task070_abductivenli_incorrect_classification",
 "Lots-of-LoRAs/task720_mmmlu_answer_generation_marketing",
 "Lots-of-LoRAs/task067_abductivenli_answer_generation",
 "Lots-of-LoRAs/task1564_triviaqa_answer_generation",
 "Lots-of-LoRAs/task270_csrg_counterfactual_context_generation",
 "Lots-of-LoRAs/task167_strategyqa_question_generation",
 "Lots-of-LoRAs/task1504_hatexplain_answer_generation",
 "Lots-of-LoRAs/task178_quartz_question_answering",
 "Lots-of-LoRAs/task277_stereoset_sentence_generation_stereotype",
 "Lots-of-LoRAs/task1315_find_range_array",
 "Lots-of-LoRAs/task1434_head_qa_classification",
 "Lots-of-LoRAs/task192_hotpotqa_sentence_generation",
 "Lots-of-LoRAs/task1157_bard_analogical_reasoning_rooms_for_containers",
 "Lots-of-LoRAs/task872_nummersense",
 "Lots-of-LoRAs/task563_discofuse_answer_generation",
 "Lots-of-LoRAs/task714_mmmlu_answer_generation_human_sexuality",
 "Lots-of-LoRAs/task1212_atomic_classification_hasproperty",
 "Lots-of-LoRAs/task495_semeval_headline_classification",
 "Lots-of-LoRAs/task1583_bless_meronym_classification",
 "Lots-of-LoRAs/task753_svamp_addition_question_answering",
 "Lots-of-LoRAs/task433_wormat_classification_profession_anit",
 "Lots-of-LoRAs/task1427_country_region_in_world",
 "Lots-of-LoRAs/task092_check_prime_classification",
 "Lots-of-LoRAs/task1285_kpa_keypoint_matching",

Figure 7: Training tasks from Lots-of-LoRAs (based on the SNI dataset) used for training the Text-to-LoRA model. The struck out names indicate removed tasks due to benchmark contamination.

Training Tasks (cont.)

~~"Lots-of-LoRAs/task333_hateval_classification_hate_en",~~
~~"Lots-of-LoRAs/task329_gap_classification",~~
~~"Lots-of-LoRAs/task398_semeval_2018_task1_tweet_joy_detection",~~
~~"Lots-of-LoRAs/task157_count_vowels_and_consonants",~~
~~"Lots-of-LoRAs/task074_corpus1_1_question_generation",~~
~~"Lots-of-LoRAs/task1508_celebrity_minimal_dob_span",~~
~~"Lots-of-LoRAs/task697_mmmlu_answer_generation_formal_logic",~~
~~"Lots-of-LoRAs/task285_imdb_answer_generation",~~
~~"Lots-of-LoRAs/task393_plausible_result_generation",~~
~~"Lots-of-LoRAs/task1147_country_currency",~~
~~"Lots-of-LoRAs/task1585_root09_hypemym_generation",~~
~~"Lots-of-LoRAs/task648_answer_generation",~~
~~"Lots-of-LoRAs/task353_casino_classification_negotiation_elicit_pref",~~
~~"Lots-of-LoRAs/task1431_head_qa_answer_generation",~~
~~"Lots-of-LoRAs/task148_afs_argument_quality_gay_marriage",~~
~~"Lots-of-LoRAs/task585_preposition_classification",~~
~~"Lots-of-LoRAs/task081_pigs_wrong_answer_generation",~~
~~"Lots-of-LoRAs/task477_cls_english_dvd_classification",~~
~~"Lots-of-LoRAs/task1582_bless_hypemym_generation",~~
~~"Lots-of-LoRAs/task365_casino_classification_negotiation_other_need",~~
~~"Lots-of-LoRAs/task284_boring_question_generation",~~
~~"Lots-of-LoRAs/task633_dbpedia_14_answer_generation",~~
~~"Lots-of-LoRAs/task093_conala_normalize_lists",~~
~~"Lots-of-LoRAs/task722_mmmlu_answer_generation_random_topic",~~
~~"Lots-of-LoRAs/task566_circa_classification",~~
~~"Lots-of-LoRAs/task1152_bard_analogical_reasoning_causation",~~
~~"Lots-of-LoRAs/task1452_location_entity_extraction_btc_corpus",~~
~~"Lots-of-LoRAs/task422_gossip_classification_questioning",~~
~~"Lots-of-LoRAs/task925_coached_conv_pref_classifier",~~
~~"Lots-of-LoRAs/task1703_ljspeech_textmodification",~~
~~"Lots-of-LoRAs/task833_poem_sentiment_classification",~~
~~"Lots-of-LoRAs/task1210_atomic_classification_madeup0f",~~
~~"Lots-of-LoRAs/task679_hope_edi_english_text_classification",~~
~~"Lots-of-LoRAs/task1203_atomic_classification_xreact",~~
~~"Lots-of-LoRAs/task1089_check_monotonic_array",~~
~~"Lots-of-LoRAs/task389_torque_generate_temporal_question",~~
~~"Lots-of-LoRAs/task378_reverse_words_of_given_length",~~
~~"Lots-of-LoRAs/task637_extract_and_sort_unique_digits_in_a_list",~~
~~"Lots-of-LoRAs/task101_reverse_and_concatenate_all_elements_from_index_i_to_j",~~
~~"Lots-of-LoRAs/task1355_sent_comp_summarization",~~
~~"Lots-of-LoRAs/task640_esnli_classification",~~
~~"Lots-of-LoRAs/task344_hybridqa_answer_generation",~~
~~"Lots-of-LoRAs/task1190_add_integer_to_list",~~
~~"Lots-of-LoRAs/task733_mmmlu_answer_generation_security_studies",~~
~~"Lots-of-LoRAs/task107_splash_question_to_sql",~~
~~"Lots-of-LoRAs/task1209_atomic_classification_objectuse",~~
~~"Lots-of-LoRAs/task505_count_all_numerical_elements_in_list",~~
~~"Lots-of-LoRAs/task1385_anli_r1_entailment",~~
~~"Lots-of-LoRAs/task1136_vocab_commonsense_mc_classification",~~
~~"Lots-of-LoRAs/task328_jigsaw_classification_insult",~~
~~"Lots-of-LoRAs/task609_sbic_potentially_offense_binary_classification",~~
~~"Lots-of-LoRAs/task413_mickey_en_sentence_perturbation_generation",~~
~~"Lots-of-LoRAs/task072_abductivenli_answer_generation",~~
~~"Lots-of-LoRAs/task1425_country_iso_numeric",~~
~~"Lots-of-LoRAs/task1451_drug_dose_extraction",~~
~~"Lots-of-LoRAs/task713_mmmlu_answer_generation_human_aging",~~
~~"Lots-of-LoRAs/task642_esnli_classification",~~
~~"Lots-of-LoRAs/task1321_country_continent",~~
~~"Lots-of-LoRAs/task454_swag_incorrect_answer_generation",~~
~~"Lots-of-LoRAs/task504_count_all_alphabetical_elements_in_list",~~
~~"Lots-of-LoRAs/task696_mmmlu_answer_generation_elementary_mathematics",~~
~~"Lots-of-LoRAs/task1429_evaluation_semantic_relation_classification",~~
~~"Lots-of-LoRAs/task1645_medical_question_pair_dataset_text_classification",~~
~~"Lots-of-LoRAs/task431_senteval_object_count",~~
~~"Lots-of-LoRAs/task1317_country_calling_code",~~
~~"Lots-of-LoRAs/task131_scan_long_text_generation_action_command_long",~~
~~"Lots-of-LoRAs/task675_google_wellformed_query_sentence_generation",~~
~~"Lots-of-LoRAs/task1158_bard_analogical_reasoning_manipulating_items",~~
~~"Lots-of-LoRAs/task1325_qa_zre_question_generation_on_subject_relation",~~
~~"Lots-of-LoRAs/task300_storycloze_order_generation",~~
~~"Lots-of-LoRAs/task1347_glue_sts-b_similarity_classification",~~
~~"Lots-of-LoRAs/task1328_qa_zre_relation_generation_from_question",~~
~~"Lots-of-LoRAs/task630_dbpedia_14_classification",~~
~~"Lots-of-LoRAs/task460_gasper_answer_generation",~~
~~"Lots-of-LoRAs/task1665_trianglecopa_question_generation",~~
~~"Lots-of-LoRAs/task1154_bard_analogical_reasoning_travel",~~
~~"Lots-of-LoRAs/task190_storycloze_order_generation",~~
~~"Lots-of-LoRAs/task617_amazonreview_category_text_generation",~~
~~"Lots-of-LoRAs/task1508_wordnet_antonyms",~~
~~"Lots-of-LoRAs/task628_xhwc_word_with_different_meaning_sentence_generation",~~
~~"Lots-of-LoRAs/task600_find_the_longest_common_substring_in_two_strings",~~
~~"Lots-of-LoRAs/task1380_quarel_correct_option_generation",~~
~~"Lots-of-LoRAs/task489_mwsc_question_generation",~~
~~"Lots-of-LoRAs/task905_hate_speech_offensive_classification",~~
~~"Lots-of-LoRAs/task065_limetravel_consistent_sentence_classification",~~
~~"Lots-of-LoRAs/task044_essential_terms_identifying_essential_words",~~
~~"Lots-of-LoRAs/task069_abductivenli_classification",~~
~~"Lots-of-LoRAs/task356_casino_classification_negotiation_self_need",~~
~~"Lots-of-LoRAs/task403_creak_commonsense_inference",~~
~~"Lots-of-LoRAs/task590_amazonfood_summary_correction_classification",~~
~~"Lots-of-LoRAs/task577_curiosity_dialogs_classification",~~
~~"Lots-of-LoRAs/task519_per_sentiment_classification",~~
~~"Lots-of-LoRAs/task1721_civil_comments_obscenity_classification",~~
~~"Lots-of-LoRAs/task351_winomt_classification_gender_identifiability_anti",~~
~~"Lots-of-LoRAs/task904_hate_speech_offensive_classification",~~
~~"Lots-of-LoRAs/task244_count_elements_in_set_union",~~
~~"Lots-of-LoRAs/task1712_poki_classification",~~
~~"Lots-of-LoRAs/task683_online_privacy_policy_text_purpose_answer_generation",~~
~~"Lots-of-LoRAs/task1197_atomic_classification_creat",~~
~~"Lots-of-LoRAs/task1592_yahoo_answers_topics_classification",~~

Training Tasks (cont.)

~~"Lots-of-LoRAs/task274_outruling_legal_classification",~~
~~"Lots-of-LoRAs/task278_stereoset_sentence_generation_antistereotype",~~
~~"Lots-of-LoRAs/task1534_daily_dialog_question_classification",~~
~~"Lots-of-LoRAs/task891_gap_coreference_resolution",~~
~~"Lots-of-LoRAs/task694_mmmlu_answer_generation_econometrics",~~
~~"Lots-of-LoRAs/task497_extract_all_numbers_from_list_in_order",~~
~~"Lots-of-LoRAs/task488_extract_all_alphabetical_elements_from_list_in_order",~~
~~"Lots-of-LoRAs/task144_subjq_question_answering",~~
~~"Lots-of-LoRAs/task1722_civil_comments_threat_classification",~~
~~"Lots-of-LoRAs/task1728_web_nlp_data_to_text",~~
~~"Lots-of-LoRAs/task291_semexval_2020_task4_commonsense_validation",~~
~~"Lots-of-LoRAs/task284_imdb_classification",~~
~~"Lots-of-LoRAs/task1288_glue_mrpc_paraphrasing",~~
~~"Lots-of-LoRAs/task128_scan_structured_text_generation_command_action_short",~~
~~"Lots-of-LoRAs/task580_socialqa_answer_generation",~~
~~"Lots-of-LoRAs/task616 cola_classification",~~
~~"Lots-of-LoRAs/task1401_obqa_sentence_generation",~~
~~"Lots-of-LoRAs/task716_mmmlu_answer_generation_jurisprudence",~~
~~"Lots-of-LoRAs/task1421_mathqa_other",~~
~~"Lots-of-LoRAs/task1656_goaqa_answer_generation",~~
~~"Lots-of-LoRAs/task1311_amazonreview_rating_classification",~~
~~"Lots-of-LoRAs/task516_senteval_conjoints_inversion",~~
~~"Lots-of-LoRAs/task593_scic_explanation_generation",~~
~~"Lots-of-LoRAs/task138_detoxifying_tms_classification_fluency",~~
~~"Lots-of-LoRAs/task1186_nre_himgo_classification",~~
~~"Lots-of-LoRAs/task1326_qa_zre_question_generation_from_answer",~~
~~"Lots-of-LoRAs/task119_semeval_geometric_mathematical_answer_generation",~~
~~"Lots-of-LoRAs/task108_contextualabusedetection_classification",~~
~~"Lots-of-LoRAs/task584_udeps_eng_fine_pos_tagging",~~
~~"Lots-of-LoRAs/task388_torque_token_classification",~~
~~"Lots-of-LoRAs/task045_miscellaneous_sentence_paraphrasing",~~
~~"Lots-of-LoRAs/task821_protoqa_question_generation",~~
~~"Lots-of-LoRAs/task1581_qcqa-perturbed_answer_generation",~~
~~"Lots-of-LoRAs/task695_mmmlu_answer_generation_electrical_engineering",~~
~~"Lots-of-LoRAs/task596_mocha_question_generation",~~
~~"Lots-of-LoRAs/task568_circa_question_generation",~~
~~"Lots-of-LoRAs/task085_unnatural_addsub_arithmetic",~~
~~"Lots-of-LoRAs/task1495_adverse_drug_event_classification",~~
~~"Lots-of-LoRAs/task1403_check_validity_date_mmddyyyy",~~
~~"Lots-of-LoRAs/task1453_person_entity_extraction_btc_corpus",~~
~~"Lots-of-LoRAs/task1201_atomic_classification_xintent",~~
~~"Lots-of-LoRAs/task923_event2mind_classifier",~~
~~"Lots-of-LoRAs/task1204_atomic_classification_hinderedby",~~
~~"Lots-of-LoRAs/task1510_evaluation_relation_extraction",~~
~~"Lots-of-LoRAs/task754_svamp_common-division_question_answering",~~
~~"Lots-of-LoRAs/task1403_check_validity_date_mmddyyyy",~~
~~"Lots-of-LoRAs/task1192_food_flavor_profile",~~
~~"Lots-of-LoRAs/task565_circa_answer_generation",~~
~~"Lots-of-LoRAs/task146_afs_argument_similarity_gun_control",~~
~~"Lots-of-LoRAs/task666_mmmlu_answer_generation_astronomy",~~
~~"Lots-of-LoRAs/task050_multirc_answerability",~~
~~"Lots-of-LoRAs/task704_mmmlu_answer_generation_high_school_government",~~
~~"Lots-of-LoRAs/task1447_drug_extraction_ade",~~
~~"Lots-of-LoRAs/task579_socialqa_classification",~~
~~"Lots-of-LoRAs/task1196_atomic_classification_oeffect",~~
~~"Lots-of-LoRAs/task267_concatenate_and_reverse_all_elements_from_index_i_to_j",~~
~~"Lots-of-LoRAs/task206_collatz_conjecture",~~
~~"Lots-of-LoRAs/task936_defeasible_nli_snli_classification",~~
~~"Lots-of-LoRAs/task323_jigsaw_classification_sexually_explicit",~~
~~"Lots-of-LoRAs/task494_review_polarity_answer_generation",~~
~~"Lots-of-LoRAs/task200_gasper_question_generation",~~
~~"Lots-of-LoRAs/task1409_dart_text_generation",~~
~~"Lots-of-LoRAs/task1313_amazonreview_polarity_classification",~~
~~"Lots-of-LoRAs/task076_splash_correcting_sql_mistake",~~
~~"Lots-of-LoRAs/task686_mmmlu_answer_generation_college_biology",~~
~~"Lots-of-LoRAs/task740_hoestq_answer_generation_quantity",~~
~~"Lots-of-LoRAs/task1199_amazonreview_question_modification_emoji",~~
~~"Lots-of-LoRAs/task1211_atomic_classification_hassubevent",~~
~~"Lots-of-LoRAs/task113_count_frequency_of_letter",~~
~~"Lots-of-LoRAs/task280_stereoset_classification_stereotype_type",~~
~~"Lots-of-LoRAs/task1551_every_nth_element_from_kth_element",~~
~~"Lots-of-LoRAs/task116_com2sense_commonsense_reasoning",~~
~~"Lots-of-LoRAs/task518_emo_different_dialogue_emotions",~~
~~"Lots-of-LoRAs/task1520_qa_srl_answer_generation",~~
~~"Lots-of-LoRAs/task079_conala_concat_strings",~~
~~"Lots-of-LoRAs/task513_argument_stance_classification",~~
~~"Lots-of-LoRAs/task1590_diplomacy_text_generation",~~
~~"Lots-of-LoRAs/task1713_convai3_sentence_generation",~~
~~"Lots-of-LoRAs/task1386_anli_r2_entailment",~~
~~"Lots-of-LoRAs/task063_first_i_elements",~~
~~"Lots-of-LoRAs/task183_rhyme_generation",~~
~~"Lots-of-LoRAs/task1447_drug_extraction_ade",~~
~~"Lots-of-LoRAs/task1771_ambigua_text_generation",~~
~~"Lots-of-LoRAs/task068_abductivenli_incorrect_answer_generation",~~
~~"Lots-of-LoRAs/task858_inquisitive_span_detection",~~
~~"Lots-of-LoRAs/task699_mmmlu_answer_generation_high_school_biology",~~
~~"Lots-of-LoRAs/task1593_yahoo_answers_topics_classification",~~
~~"Lots-of-LoRAs/task700_mmmlu_answer_generation_high_school_chemistry",~~
~~"Lots-of-LoRAs/task363_sst2_polarity_classification",~~
~~"Lots-of-LoRAs/task121_zest_text_modification",~~
~~"Lots-of-LoRAs/task190_snli_classification",~~
~~"Lots-of-LoRAs/task1168_brown_coarse_pos_tagging",~~
~~"Lots-of-LoRAs/task195_sentiment140_classification",~~
~~"Lots-of-LoRAs/task1723_civil_comments_sexuallyexplicit_classification",~~
~~"Lots-of-LoRAs/task1449_disease_entity_extraction_bc5cdi_dataset",~~
~~"Lots-of-LoRAs/task1418_mathqa_gain",~~
~~"Lots-of-LoRAs/task1398_obqa_question_generation",~~
~~"Lots-of-LoRAs/task893_gap_fill_the_blank_reference_resolution",~~
~~"Lots-of-LoRAs/task326_jigsaw_classification_obscene",~~

Figure 8: Training tasks from Lots-of-LoRAs (based on the SNI dataset) used for training the Text-to-LoRA model. The stricken out names indicate removed tasks due to benchmark contamination.

Training Tasks (cont.)

"Lots-of-LoRAs/task1194_kth_largest_element",
 "Lots-of-LoRAs/task102_commonsense_sentence_generation",
 "Lots-of-LoRAs/task145_afs_argument_similarity_death_penalty",
 "Lots-of-LoRAs/task1338_peixian_equality_evaluation_corpus_sentiment_classifier",
 "Lots-of-LoRAs/task391_causal_relationship",
 "Lots-of-LoRAs/task176_break_decompose_questions",
 "Lots-of-LoRAs/task319_stereoset_classification_profession",
 "Lots-of-LoRAs/task369_casino_classification_negotiation_vouch_fair",
 "Lots-of-LoRAs/task866_conv_ai_2_classification",
 "Lots-of-LoRAs/task1729_personachat_generate_next",
 "Lots-of-LoRAs/task761_app_review_classification",
 "Lots-of-LoRAs/task1320_country_domain_id",
 "Lots-of-LoRAs/task1596_event2mind_text_generation_2",
 "Lots-of-LoRAs/task1601_webquestions_answer_generation",
 "Lots-of-LoRAs/task615_moviesqa_answer_generation",
 "Lots-of-LoRAs/task1283_hrngo_quality_classification",
 "Lots-of-LoRAs/task638_multi_woz_classification",
 "Lots-of-LoRAs/task607_sbic_intentional_offense_binary_classification",
 "Lots-of-LoRAs/task692_mmmlu_answer_generation_computer_security",
 "Lots-of-LoRAs/task698_amazonfood_rating_classification",
 "Lots-of-LoRAs/task129_scan_long_text_generation_action_command_short",
 "Lots-of-LoRAs/task1200_atomic_classification_xeffect",
 "Lots-of-LoRAs/task1486_cell_extraction_anem_dataset",
 "Lots-of-LoRAs/task1406_kth_smallest_element",
 "Lots-of-LoRAs/task583_udeps_eng_coarse_pos_tagging",
 "Lots-of-LoRAs/task094_conala_calculate_mean",
 "Lots-of-LoRAs/task1731_quantz_question_answering",
 "Lots-of-LoRAs/task308_jeopardy_answer_generation_all",
 "Lots-of-LoRAs/task664_mmmlu_answer_generation_abstract_algebra",
 "Lots-of-LoRAs/task163_count_words_ending_with_letter",
 "Lots-of-LoRAs/task127_scan_long_text_generation_action_command_all",
 "Lots-of-LoRAs/task736_mmmlu_answer_generation_virology",
 "Lots-of-LoRAs/task379_agnews_topic_classification",
 "Lots-of-LoRAs/task390_gap_answer_generation",
 "Lots-of-LoRAs/task1599_smcalfow_classification",
 "Lots-of-LoRAs/task875_emotion_classification",
 "Lots-of-LoRAs/task1214_atomic_classification_xwant",
 "Lots-of-LoRAs/task933_wiki_auto_style_transfer",
 "Lots-of-LoRAs/task1319_country_by_barcode_prefix",
 "Lots-of-LoRAs/task456_matres_intention_classification",
 "Lots-of-LoRAs/task1189_check_char_in_string",
 "Lots-of-LoRAs/task1857_goose_question_generation",
 "Lots-of-LoRAs/task517_emo_classify_emotion_of_dialogue",
 "Lots-of-LoRAs/task385_socialqa_incorrect_answer_generation",
 "Lots-of-LoRAs/task472_haspart_classification",
 "Lots-of-LoRAs/task047_miscellaneous_answering_science_questions",
 "Lots-of-LoRAs/task1533_daily_dialog_formal_classification",
 "Lots-of-LoRAs/task249_enhanced_wsc_pronoun_disambiguation",
 "Lots-of-LoRAs/task095_conala_max_absolute_value",
 "Lots-of-LoRAs/task1479_organization_entity_extraction_btc_corpus",
 "Lots-of-LoRAs/task724_mmmlu_answer_generation_moral_scenarios",
 "Lots-of-LoRAs/task1418_bless_semantic_relation_classification",
 "Lots-of-LoRAs/task507_position_of_all_numerical_elements_in_list",
 "Lots-of-LoRAs/task043_essential_terms_answering_incomplete_questions",
 "Lots-of-LoRAs/task1394_meta_woz_task_classification",
 "Lots-of-LoRAs/task149_afs_argument_quality_death_penalty",
 "Lots-of-LoRAs/task1156_bard_analogical_reasoning_tools",
 "Lots-of-LoRAs/task1598_nyc_long_text_generation",
 "Lots-of-LoRAs/task322_jigsaw_classification_threat",
 "Lots-of-LoRAs/task1714_convai3_sentence_generation",
 "Lots-of-LoRAs/task727_mmmlu_answer_generation_prehistory",
 "Lots-of-LoRAs/task609_jigsaw_answer_generation",
 "Lots-of-LoRAs/task1704_ljspeech_textmodification",
 "Lots-of-LoRAs/task077_splash_explanation_to_sql",
 "Lots-of-LoRAs/task1390_wscfixed_coreference",
 "Lots-of-LoRAs/task667_mmmlu_answer_generation_business_ethics",
 "Lots-of-LoRAs/task1724_civil_comments_insult_classification",
 "Lots-of-LoRAs/task1088_array_of_products",
 "Lots-of-LoRAs/task1087_two_number_sum",
 "Lots-of-LoRAs/task550_disclose_sentence_generation",
 "Lots-of-LoRAs/task892_gap_reverse_coreference_resolution",
 "Lots-of-LoRAs/task674_google_wellformed_query_sentence_generation",
 "Lots-of-LoRAs/task1280_holloway_completion",
 "Lots-of-LoRAs/task509_collate_of_all_alphabetical_and_numerical_elements_in_list_separately",
 "Lots-of-LoRAs/task346_hybridqa_classification",
 "Lots-of-LoRAs/task769_qed_summarization",
 "Lots-of-LoRAs/task1205_winogrande_easy_question_generation",
 "Lots-of-LoRAs/task1322_country_government_type",
 "Lots-of-LoRAs/task341_winomt_classification_gender_ant",
 "Lots-of-LoRAs/task290_tellmewhy_question_answerability",
 "Lots-of-LoRAs/task335_hateval_classification_aggressive_en",
 "Lots-of-LoRAs/task879_schema_guided_dstc8_classification",
 "Lots-of-LoRAs/task861_asdiv_addsub_question_answering",
 "Lots-of-LoRAs/task594_sciq_question_generation",
 "Lots-of-LoRAs/task066_limettravel_binary_consistency_classification",
 "Lots-of-LoRAs/task162_count_words_starting_with_letter",
 "Lots-of-LoRAs/task1584_evaluation_meronym_classification",
 "Lots-of-LoRAs/task1622_disfl_qa_text_modication",
 "Lots-of-LoRAs/task247_dream_answer_generation"

Figure 9: Training tasks from Lots-of-LoRAs (based on the SNI dataset) used for training the Text-to-LoRA model. The stricken out names indicate removed tasks due to benchmark contamination.

Validation Tasks

"Lots-of-LoRAs/task035_winogrande_question_modification_person",
 "Lots-of-LoRAs/task039_qasc_find_overlapping_words",
 "Lots-of-LoRAs/task1557_jfleg_answer_generation",
 "Lots-of-LoRAs/task202_mnli_contradiction_classification",
 "Lots-of-LoRAs/task304_numeric_fused_head_resolution",
 "Lots-of-LoRAs/task362_spolin_yesand_prompt_response_sub_classification",
 "Lots-of-LoRAs/task614_glucose_cause_event_detection",
 "Lots-of-LoRAs/task701_mmmlu_answer_generation_high_school_computer_science",
 "Lots-of-LoRAs/task706_mmmlu_answer_generation_high_school_mathematics",
 "Lots-of-LoRAs/task710_mmmlu_answer_generation_high_school_statistics",
 "Lots-of-LoRAs/task726_mmmlu_answer_generation_philosophy",

Figure 10: Validation tasks used during the training of the Text-to-LoRA model.

J.1. Benchmark Details

Every benchmark used in the experiments is publicly available in HuggingFace dataset space. We evaluate the models on the benchmarks detailed as follows.

J.1.1. GSM8K

We evaluate the models on the test split, using chain-of-thought response pre-filling: *"Let's think step by step."*

J.1.2. HUMAN EVAL AND MBPP

We use the `evalplus` library (Liu et al., 2023) for coding evaluation. For both MBPP and HumanEval, we use the following response pre-fill: ``python

J.2. Question-Answering Tasks

The rest of the benchmarks are question-answering based tasks. In these tasks, we do not use response-prefilling. Instead, each task has a specific instruction template shown in Listing 4.

K. Utilizing Full Adaptation Matrix vs Low-Rank Matrices

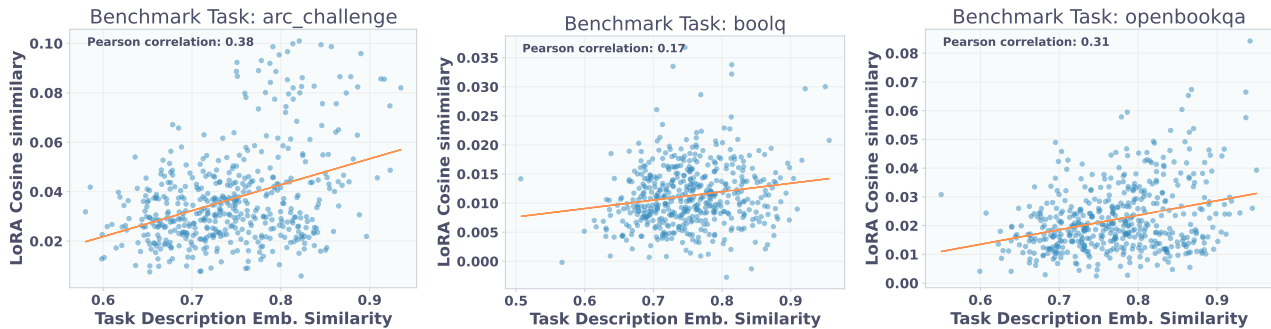


Figure 11: Each plot shows the similarity between a benchmark LoRA adapter and 479 SNI-trained adapters in the ΔW weight space. There is a positive correlation between the two variables indicated by small positive Pearson correlation coefficients.

Similar to Figure 6, Figure 11 shows the similarity of SNI adapters to benchmark-specific adapters, but instead of using the concatenation of flattened A and B matrices, we use flattened ΔW instead. With the change, we find a positive correlation between the task embedding similarity and the adapter similarity in the weight space. This is likely because, for a given ΔW matrix, there are many possible permutations of low-rank matrices A and B . This suggests that if we compute the reconstruction loss in the full adaptation matrix space, reconstruction-trained T2L could generalize better. However, we empirically find that it does not outperform T2L trained to reconstruct low-rank matrices at zero-shot LoRA generation.

L. Generating Task Descriptions with a Foundation Language Model

We automate task description generation for each task by leveraging powerful closed-source language models (Achiam et al., 2023). We query GPT-4o mini with carefully constructed prompts that incentivize diversity to facilitate downstream generalization. In particular, we generate 200 descriptions per task by querying the model 10 times, each time asking for 20 descriptions given randomly sampled five question-answer pairs from the task. We leverage in-context learning by providing examples of question-answer pairs with matching descriptions. Finally, we also designed our prompts to avoid overly verbose responses and unnecessary information, such as explicit mentions of answer formats and additional instructions. We use the generated descriptions for the training and benchmark tasks. Figure 12 shows the exact prompt used for querying GPT-4o mini for task descriptions.

System message

You are a creative and helpful assistant.

Prompt

Given the following question-response pairs, please give a short description of the task describing what the task is.

{IN CONTEXT EXAMPLES}

Now, you must describe the task based on the following question-response pairs.

{5 sampled question-answer pairs}

Please use the information in the question-answer pairs and example description and come up with several descriptions that explain the task. Each description should be written in plain text, with the following format.

Description 1: DESCRIPTION_1

Description 2: DESCRIPTION_2

...

You should also be creative and vary the structure and the length of the descriptions such that they'll be diverse and cover various writing styles. You should ignore the specific question-answer pairs and focus on the high-level concept and topic of the task in general.

****DO NOT**** describe that there are multiple choice options or the format of the answer.

****DO NOT**** include the answer format, e.g., 'choose the correct option', 'answer with only one word', etc.

****DO NOT**** describe how to answer the question, but rather what the task is about and the skills and knowledge required.

You can include reasoning steps that should be used to reach the expected answer.

Response with 20 descriptions. Use simple words and please be clear and diverse in your descriptions.

In-context examples

Here are some examples of the structure of the task of describing a task based on question-response pairs.

Example question-answer pair: 1

Input

You are given a question on high school macroeconomics. You are also given 4 answer options (associated with 'A', 'B', 'C', 'D'), out of which only one is correct. You need to answer the question by selecting the correct option. You should only answer with the choice letter, not the whole answer.

Input: Allocative efficiency (A) means that no inferior products will be produced. (B) implies that the economy's output is distributed evenly. (C) means that those who work hardest will get more. (D) implies that resources are used to produce the goods and services society desires in just the right amounts.

Output:

Expected output

D

Plausible descriptions

Description 1: Your job is to analyze the provided question about economics. Use your understanding of economic principles to guide your choice.

Description 2: Utilize your economic understanding to determine which choice is right. The correct answer will be the one that best aligns with economic principles.

Example question-answer pair: 2

Input

In this task, you are given a country name and you need to return the capital city of the given country.

Input: Senegal

Output:

Expected output

Dakar

Plausible descriptions

Description 1: Given the name of a country, your job is to provide its capital city.

Description 2: For each country listed, determine and state its capital city. This requires familiarity with global locations and capitals.

Figure 12: The prompt template used to query GPT-4o mini for task descriptions.

M. Example of Task Descriptions

Here, we provide examples of task descriptions used in the experiments.

Training descriptions

sni_cosmosqa_passage_inappropriate_binary

- Assess whether the given passage contains any elements that are unsuitable or illogical. Contextual understanding is key to making your evaluation.
- Look closely at the information provided in the context and determine its appropriateness or nonsensical nature based on logical reasoning.
- Assess given contexts critically, marking whether they hold inappropriate content or convey meaning in a way that is difficult to comprehend.

sni_winomt_classification_gender_identifiability_anti

- In this task, you will distinguish between identifiable and unidentifiable gender references in sentences featuring different professions.
- Your task consists of evaluating professional descriptions within sentences and determining if their respective genders can be classified as clearly identifiable or obscure.
- Engage with sentences that present two different professions, paying attention to pronouns that could reveal or obscure the gender of the highlighted role.

sni_kth_largest_element

- In this task, you are required to dissect a set of integers and identify which one corresponds to the kth position when sorted by size. Knowledge of ascending order and magnitude awareness are pivotal.
- Your mission here is to discover which number holds the kth place when considering size among others in a list. Practicing sorting and prioritization will be beneficial.
- The job is to pick out the kth greatest number from a list of integers, which means reevaluating them according to their increasing or decreasing order.

Figure 13: Examples of training descriptions from three SNI training tasks.

Evaluation descriptions

boolq

- Analyze the given details about various subjects, including movies, sports, and television shows. Your role is to confirm whether certain claims are true or false.
- Your task is to determine the truthfulness of specific statements based on the provided background information. This requires careful reading and comprehension of the content.
- The goal is to evaluate factual claims made in relation to highlighted texts. You will need to discern whether the statements align with the information provided.

gsm8k

- You will be tasked with interpreting mathematical situations described in words. The goal is to use logical reasoning and calculations to determine the numerical answers based on the context provided.
- This task challenges your problem-solving abilities through mathematical reasoning. You must carefully read each scenario and systematically work through the data to compute the final outcome.
- Your role is to engage with practical math scenarios presented as questions. The task requires translating textual data into numerical operations that will lead you to the final solution.

humaneval

- Engage in building distinct functions that meet the requirements of various presented problems, honing your ability to translate problem statements into logical code. Utilize structured thinking to implement efficient solutions.
- You are tasked with generating specific solutions in Python by interpreting problem descriptions associated with tasks like counting odds or validating inputs. Recognizing patterns and leveraging programming techniques will be beneficial.
- This task focuses on developing algorithms in Python for specific scenarios, such as counting characters, assessing conditions between numbers, or converting integers into a different format. Critical thinking and algorithmic design will be important.

Figure 14: Task descriptions of the benchmark tasks: boolq, gsm8k, and humaneval.

Evaluation descriptions

mbpp

- Your challenge is to solve a series of problems by writing functions in Python. These problems require handling lists and strings, allowing you to showcase your proficiency in coding while addressing practical programming scenarios.
- You will be tasked with creating various Python functions that tackle programming challenges. The exercises will test your ability to manipulate data structures, search for patterns, and implement checks on numerical products.
- The goal is to develop Python functions that perform designated operations on lists and strings. This requires a solid grasp of logical reasoning and the ability to apply relevant algorithms in your code.

winogrande

- In this exercise, you need to read short narratives and discern which person or object fits best within the context of the sentence.
- This task requires synthesizing information from concise textual scenarios to identify crucial elements that drive the narrative forward.
- The goal is to evaluate descriptions and select the entity that best aligns with the sentiments or actions presented in the scenario.

piqa

- You will explore practical questions and select an answer that presents a logical and widely accepted approach to solve a given problem or complete a task successfully.
- Analyze the provided scenarios where practical advice or solutions are required, focusing on selecting the most commonly used or convenient method.
- Given a question related to common tasks, your responsibility is to discern which proposed solution aligns with typical practices or makes the task easier to achieve.

Figure 15: Task descriptions of the benchmark tasks: mbpp, winogrande, piqa

Evaluation descriptions

hellaswag

- This task revolves around completing an unfinished text by selecting an ending that matches its tone and context. It requires you to think critically about how narratives develop and conclude effectively.
- This task asks you to select a suitable conclusion for an unfinished narrative or instructional content. It tests your comprehension and reasoning skills as you assess how well each option aligns with the given text.
- Your task involves completing an incomplete passage by selecting the ending that logically continues the context provided. This requires reading comprehension and the ability to infer meaning from a text.

arc_easy

- Your job is to discern which information best answers a posed question, focusing on practical examples and scientific principles. This requires a strong grasp of underlying concepts in ecology or physics.
- You will analyze questions that explore important connections such as environmental issues or animal adaptations. Utilize your background knowledge to evaluate and select the most fitting answer.
- This task involves selecting answers that reflect accurate relationships or effects seen in nature or society. You will need to sort through potential choices critically to find the appropriate one.

arc_challenge

- This task is about analyzing questions which examine your grasp of scientific ideas. You must connect conceptual knowledge with practical examples from geology, ecology and environmental changes.
- The objective here is to evaluate various scientific scenarios and infer the most logical explanations or definitions based on established knowledge. This task will strengthen your analytical and reasoning skills in the context of natural science.
- Your role is to interpret questions focusing on earth science and biological interactions. This demands a clear understanding of relevant processes, such as decomposition, weathering, and species adaptation.

Figure 16: Task descriptions of the benchmark tasks: hellaswag, arc_easy, arc_challenge

Evaluation descriptions**openbookqa**

- Analyze the provided statements carefully and determine which one best fits into the context of the passage. This requires comprehension skills and the ability to make logical inferences.
- Consider each option in relation to what is presented in the input. Discern which one logically completes or responds accurately to the notion being expressed.
- Here, you'll be presented with different statements, and your role is to decide which one appropriately complements or responds to a scenario. This process involves critical analysis and synthesis of information.

Figure 17: Task descriptions of the benchmark tasks: openbookqa

Random descriptions

- dogs;cats;bananas;
- 7@9.qwepra#/.sd,s'2OC^039u#rdagjbl
- #####

Figure 18: Random descriptions

N. Scaling Number of Descriptions per Task

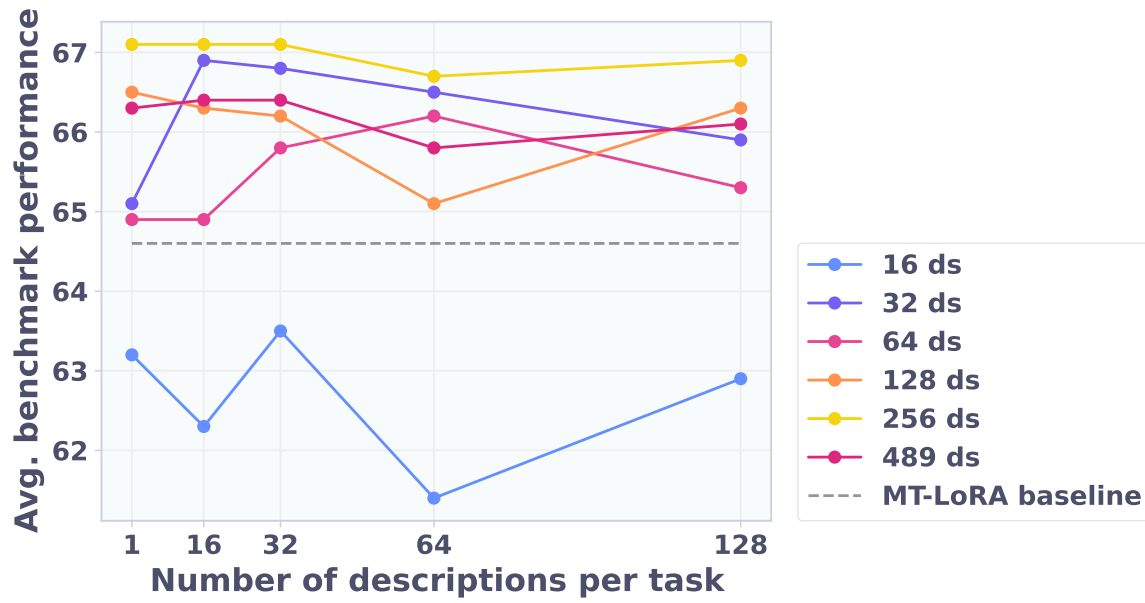


Figure 19: Zero-shot benchmark performance of SFT-trained T2L with varying number of descriptions per training task.

Figure 19 shows mixed results on the benchmark performance when varying the number of descriptions per training task. For consistency, we always train T2L with 128 descriptions per training task.


```
Hypermod: HyperModulator(  
  (task_encoder): TaskEncoder(  
    (mlp): Sequential(  
      (0): Linear(in_features=1024, out_features=64, bias=True)  
      (1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)  
    )  
  )  
  (layer_depth_encoder): Sequential(  
    (0): Embedding(32, 32)  
    (1): LayerNorm((32,), eps=1e-05, elementwise_affine=True)  
  )  
  (layer_type_encoder): Sequential(  
    (0): Embedding(2, 32)  
    (1): LayerNorm((32,), eps=1e-05, elementwise_affine=True)  
  )  
  (mixer): Sequential(  
    (0): Linear(in_features=128, out_features=512, bias=True)  
    (1): SiLU()  
    (2): Dropout(p=0.05, inplace=False)  
    (3): Linear(in_features=512, out_features=128, bias=True)  
    (4): SiLU()  
    (5): Dropout(p=0.05, inplace=False)  
  )  
  (mlp1): MLPResidualBlock(  
    (mlp): Sequential(  
      (0): LayerNorm((128,), eps=1e-05, elementwise_affine=True)  
      (1): Linear(in_features=128, out_features=512, bias=True)  
      (2): SiLU()  
      (3): Dropout(p=0.05, inplace=False)  
      (4): Linear(in_features=512, out_features=128, bias=True)  
      (5): SiLU()  
      (6): Dropout(p=0.05, inplace=False)  
    )  
  )  
  (mlp2): MLPResidualBlock(  
    (mlp): Sequential(  
      (0): LayerNorm((128,), eps=1e-05, elementwise_affine=True)  
      (1): Linear(in_features=128, out_features=512, bias=True)  
      (2): SiLU()  
      (3): Dropout(p=0.05, inplace=False)  
      (4): Linear(in_features=512, out_features=128, bias=True)  
      (5): SiLU()  
      (6): Dropout(p=0.05, inplace=False)  
    )  
  )  
  (mlp3): Sequential(  
    (0): LayerNorm((128,), eps=1e-05, elementwise_affine=True)  
    (1): Linear(in_features=128, out_features=512, bias=True)  
    (2): SiLU()  
    (3): Dropout(p=0.05, inplace=False)  
    (4): Linear(in_features=512, out_features=512, bias=True)  
    (5): SiLU()  
  )  
)
```

Listing 2: Detailed backbone architecture.

```

(AB_emb): ParameterDict(
  (q_proj): Object of type: ParameterDict
  (v_proj): Object of type: ParameterDict
  (q_proj): ParameterDict(
    (A): Parameter containing: [torch.cuda.FloatTensor of size 128]
    (B): Parameter containing: [torch.cuda.FloatTensor of size 128]
  )
  (v_proj): ParameterDict(
    (A): Parameter containing: [torch.cuda.FloatTensor of size 128]
    (B): Parameter containing: [torch.cuda.FloatTensor of size 128]
  )
)

(rank_emb): Sequential(
  (0): Embedding(8, 128)
  (1): LayerNorm((128,)), eps=1e-05, elementwise_affine=True
)

```

Listing 3: Detailed A/B and rank embedding of T2L.

```

OQA_TEMPLATE = (
  "Complete the following passage or answer the question by choosing the correct choice.\n\n"
  "{question_stem}\n\n"
  "{choices[label][0]}: {choices[text][0]}\n{choices[label][1]}: {choices[text][1]}\n"
  "{choices[label][2]}: {choices[text][2]}\n{choices[label][3]}: {choices[text][3]}\n\n"
  "You must respond with the letter corresponding to the correct choice (A,B,C,D) "
  "without any explanation."
)

ARC_TEMPLATE = (
  "Answer the question below by choosing the correct choice.\n\n"
  "{question}\n\n"
  "{choices[label][0]}: {choices[text][0]}\n{choices[label][1]}: {choices[text][1]}\n"
  "{choices[label][2]}: {choices[text][2]}\n{choices[label][3]}: {choices[text][3]}\n\n"
  "You must respond with the letter corresponding to the correct choice without any explanation."
)

HSWAG_TEMPLATE = (
  "You are provided with an incomplete passage below as well as 4 choices of continuation "
  "with only one of them being the correct ending. "
  "Treat the endings as being labelled 0, 1, 2, 3 in order.\n\n"
  "Passage: {ctx}\n\n"
  "0: {endings[0]}\n"
  "1: {endings[1]}\n"
  "2: {endings[2]}\n"
  "3: {endings[3]}\n\n"
  "You must respond with the only number corresponding to the correct ending (0,1,2,3) "
  "for the passage without any explanation."
)

PIQA_TEMPLATE = (
  "Choose the option that either answers the question, completes the sentence, "
  "or solves the problem. "
  "Pay attention to the properties of the objects in the question and how they interact with "
  "each other. "
  "If both options are correct, choose the one that is more convenient or more common. "
  "\n\n""{goal}""\n\n0: {sol1}\n1: {sol2}\n\n"
  "You must respond with either `0` or `1` without any explanation."
)

WINOGRANDE_TEMPLATE = (
  "Given the following situation:\n\n{sentence}\n\nWhich option is correct?\n\n"
  "Option 1: {option1}\nOption 2: {option2}\n\n"
  "You must respond with either `1` or `2` without any explanation."
)

BOOLQ_TEMPLATE = (
  "{passage}\n\nQuestion: {question}?\n\nPlease answer with either `true` or `false` "
  "without any explanation."
)

```

Listing 4: Instruction templates of QA-based benchmark tasks.